分布与并行数据库

分布式查询优化

主讲教师：鄂金龙
中国人民大学信息学院

1.分布式查询处理与优化

- 查询处理基础
- 查询处理器
- 查询分解
- 数据局部化
- 片段查询的优化

查询处理基础

优化目标

优化就是寻找执行代价（费用和时间）最小的查询执行策略，使系统执行效率降到最低。**优化的目标**就是指局部执行代价和网络传输代价的和最小。

- **局部执行代价**：主要指输入/输出次数（I/O代价）及CPU处理代价。
- **网络传输代价**：主要指传输启动代价和数据传输代价。

查询处理基础

执行策略

我们以一个例子来说明选择何种执行策略。

例: 求选修2号课程的学生姓名

- SQL表示:

```
select Sname
from Students, SC
where Students.Sno=SC.Sno and Cno='2';
```

- 关系代数表示:

$Q1 = \pi_{sname}(\sigma_{Students.Sno=SC.Sno\ and\ Cno='2'}(Students \times SC))$

$Q2 = \pi_{sname}(\sigma_{Cno='2'}(Students \bowtie SC))$

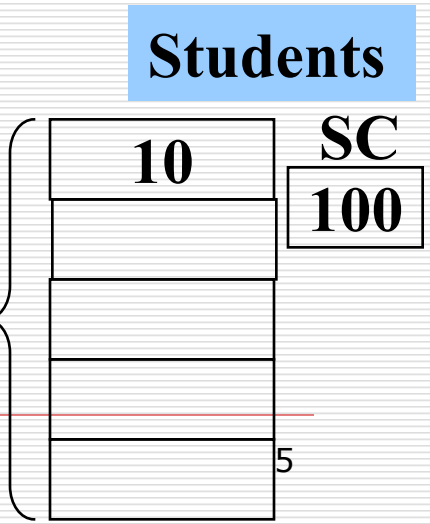
$Q3 = \pi_{sname}(Students \bowtie \sigma_{Cno='2'}(SC))$

$Q1=\pi_{sname}(\sigma_{Students.Sno=SC.Sno\ and\ Cno='2'}(Students\times SC))$

查询处理基础

代价计算

- **Q1**代价计算(仅考虑I/O代价)
  - ✓计算广义笛卡尔积代价
    - -假定: 在内存中, 存放5块Students元组和一块SC元组, 一块可以装10个Students元组或100个SC元组.
    - -假定: Students有1000个元组,SC有10000个元组, 其中选2号课程的有50个元组
    - -数据只有读到内存才能进行连接



Q1= $\pi_{sname}(\sigma_{Students.Sno=SC.Sno\ and\ Cno='2'}(Students \times SC))$

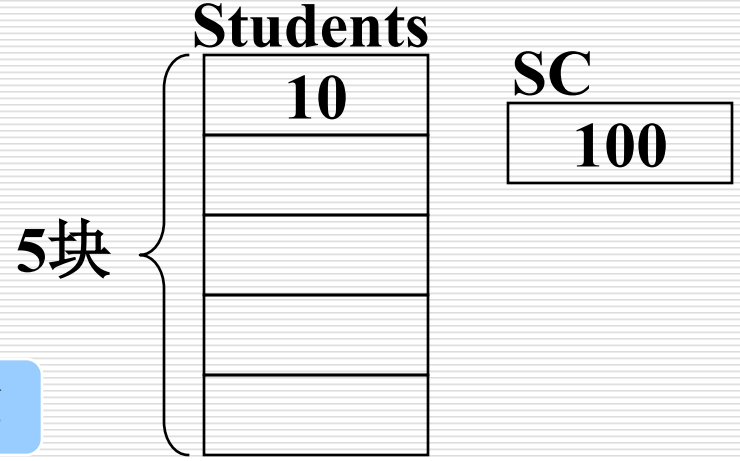
查询处理基础

- ✓ 通过读取块数计算I/O代价

读取块数计算方法:

Students 1000个元组

SC 10000个元组



Student读入内存的次数

- 读取总块数:

SC块数

Student块数

$$\frac{1000}{10}+\frac{1000}{10\times5}\times\frac{10000}{100}=100+20\times100=2100$$

- 若每秒读写20块, 则花费:

$$\frac{2100}{20}=105s$$

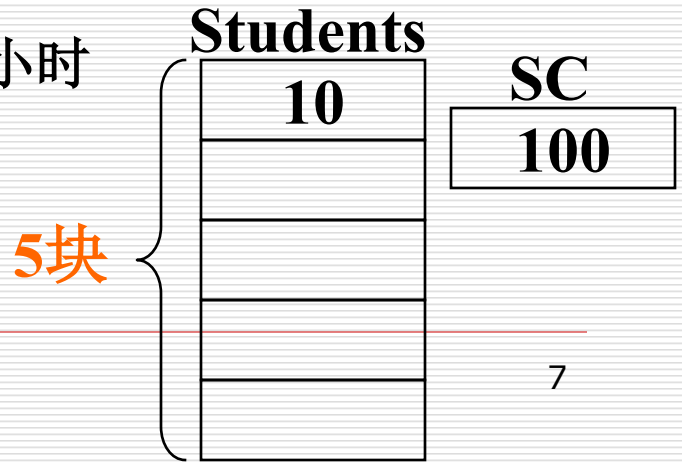
$Q1=\pi_{sname}(\sigma_{Students.Sno=SC.Sno\ and\ Cno='2'}(Students\times SC))$

查询处理基础

- ✓ 条件:Students 1000个元组， SC 10000个元组
  - 笛卡尔积计算后的元组个数为: $10^3\times10^4=10^7$
  - 连接后的中间结果内存放不下， 需暂时写到外存
  - 若每块可装10个，完成笛卡尔积后的元组， 则写这些元组需: $(10^7/10)/20=5\times10^4$s
  - 选择操作: 读回需5×10^4s, 假设选择后剩50个元组，均可放在内存
  - 投影操作: 0s
  - 查询共花费:

    $105+2\times5\times10^4\approx10^5$ s $\approx$ 28小时

每秒读写20块



Q2= $\pi_{sname}(\sigma_{Cno='2'}(Students \infty SC))$

查询处理基础

- **Q2代价计算(仅考虑I/O代价)**
  - ✓ 计算自然连接代价
    - 也要把数据读到内存进行连接, 但连接结果比笛卡尔积要小得多
    - 读取块数依然为: $\frac{1000}{10}+\frac{1000}{10\times5}\times\frac{10000}{100}=100+20\times100=2100$

      花费为**2100/20=105s**
    - 假设连接结果大小为**10^4**个元组, 写到外存需:

$(10^4$ /10)/20=50s

每秒读20块

Q2= $\pi_{sname}(\sigma_{Cno='2'}(Students \infty SC))$

查询处理基础

- 读取自然连接结果， 执行选择运算， 需50s， 选择结果均可放在内存
- 投影运算：0s
- 总花费为：105+50+50=205s ≈ **3.4分钟**

Q3= $\pi_{sname}(Students \infty \sigma_{Cno='2'}(SC))$

■ Q3代价计算(仅考虑I/O代价)

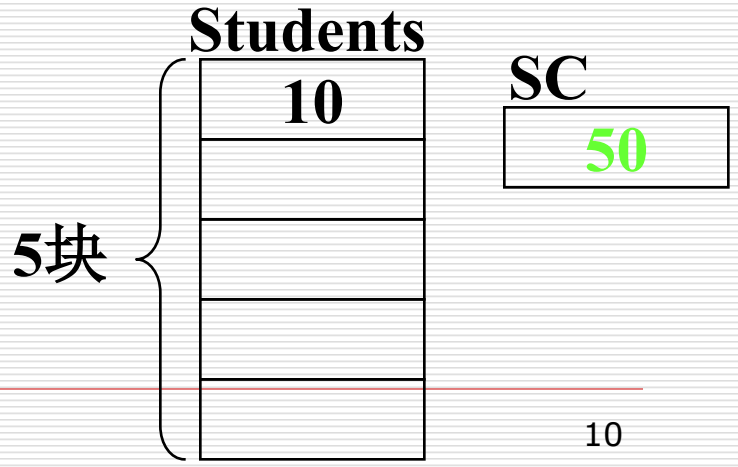
✓ 计算对SC做选择运算的代价

- 需读SC到内存进行选择运算
- 读SC块数为：10000/100=100
- 花费为：100/20=5s
- 选择结果为50个SC元组，均可放在内存

Q3= π_{sname}(Students ∞ $\sigma_{Cno='2'}$(SC))

查询处理基础

- ✓计算和Students自然连接的代价
  - 需读Students到内存进行连接运算
  - 读Students块数为:
    1000/10=100
  - 花费为: 100/20=5s
  - 连接结果为50个元组, 均可放在内存
  - 投影运算: 0s
- ✓总花费: 5+5=10s



查询处理基础

上述三种策略看，虽然都能实现所要完成的功能，但三种实现方法所需的代价却相差很大。

在分布式系统中，对于多用户、多应用需求的复杂任务，采用不同的实现策略会相差更大，将直接影响整个系统整体性能。

因此，需要确定出一种执行代价最小的查询执行策略。

查询处理基础

优化内容

优化内容体现如下几点：

- 执行运算的次序。
- 执行每种运算的方法。如上例，不同方法代价不同。
- 所访问的副本场地。如：选择就近的场地，节约传输代价。
- 执行运算的场地的选择，使总的传输代价或总代价最低。

综合考虑，确定出一种执行代价最小的查询执行策略。

查询处理基础

- 用户或应用看到的是全局关系组成的全局数据库，用户通过查询语言（通常用**SQL**语言）来表达全局查询。之后，由系统将其转换成等价的关系表达式内部表示，为描述关系的操作序列，提出一种称为查询树的内部表示方法。
- 关系代数
  - 一元运算

    u：σ（选择）**/Π**（投影）
  - 二元运算

    b：∞（连接）**/X**（笛卡儿积）**/∪**（并）**/∩**（交）**/-**（差）**/∝**（半连接）

查询处理基础

等价变换

重复律：$uR \equiv uuR$

交换律：$u_1u_2R \equiv u_2u_1R$

分配律：$u(RbS) \equiv (uR)b(uS)$

结合律：$Rb_1(Sb_2T) \equiv (Rb_1S)b_2T$

提取律：$(uR)b(uS) \equiv u(RbS)$

其中：R、S、T为关系，u_1、u_2、u为一元运算符，b_1、b_2、b为二元运算符。

机械工业出版社
CHINA MACHINE PRESS

查询处理基础

□查询树

在查询树中，叶子表示关系，中间节点表示运算，前序遍历关系表示运算次序。

■定义：

ROOT=T

T=R/（T）/TbT/uT

u=σ_F/Π_A

b=∞/X/∪/∩/-/∝

σ_F（R）===》 σ_F

R

π_A（R）===》 π_A

R

R∞S ===》 ∞

R S

R∪S ===》 ∪

R S

查询处理基础

□ 举例：设有一供应关系数据库，有供应者和供应两关系，如下：

供应者：**SUPPLIER{SNO，SNAME，AREA}**

供应者编号　供应者姓名　供应者所属地域

供应：**SUPPLY{SNO，PNO，QTY}**

供应者编号　零件号　数量

查询要求：找出地域在“北方”供应**100**号零件的供应商的信息。

SQL查询语句：**SELECT SNO，SNAME**
FROM SUPPLIER，SUPPLY
WHERE AREA=”北方”AND
PNO=100 AND
SUPPLIER.SNO=SUPPLY.SNO

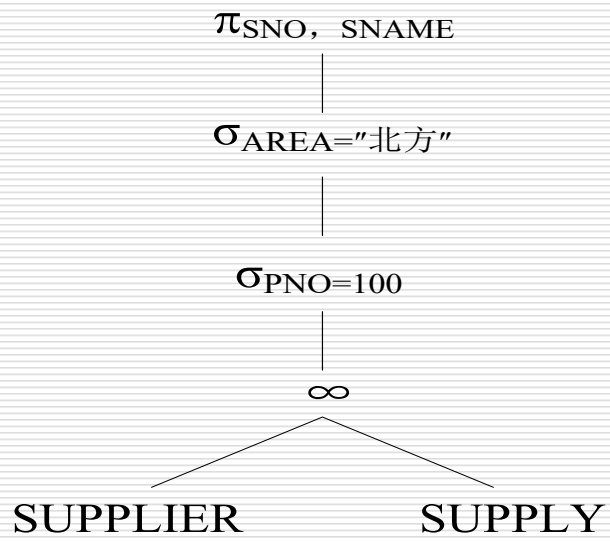
查询处理基础

□举例

等价的关系表达式：

Q1：$\Pi_{SNO,SNAME}(\sigma_{AREA="北方"}(\sigma_{PNO=100}(SUPPLIER \infty SUPPLY)))$

查询树：



查询处理器

- 影响查询处理效率的因素有：网络传输代价（数据量和延迟等）、局部I/O代价及CPU处理代价等，但主要由网络通信代价和局部I/O代价来衡量。不同的分布式数据库系统可能对评估查询处理的传输代价和I/O代价的侧重不同。
- 为提高查询的效率，在查询处理过程中还要进行优化处理。
- 查询优化就是确定出一种执行代价最小的查询执行策略或寻找相对较优的操作执行步骤。一般采用多级优化的策略。本章介绍全局查询的处理与优化。

查询处理器

- 查询语言：

SQL–非过程化的关系数据库语言，是元组演算语言

■元组演算：$\{t|F(t)\}$

其中，t:元组变量，F(t):well-formed formula(wff),

✓例如: 查询所有 managers 的 no. 和 name。

$$\{t(ENO, ENAME)|t \in E \wedge t(TITLE) =" MANAGER"\}$$

■域演算：$\{x_1, x_2, \ldots x_n | F(x_1, x_2, \ldots, x_n)\}$

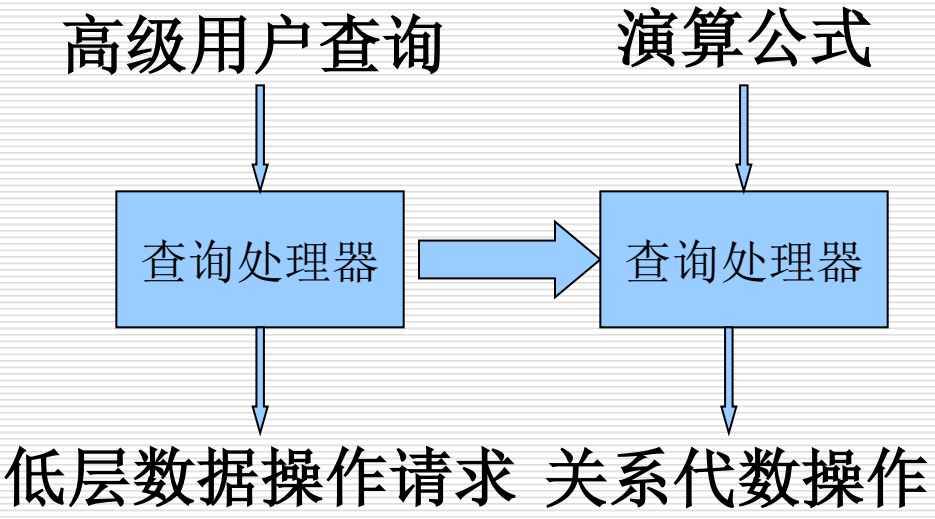
其中 x_1, x_2, x_n : 域变量, $F(x_1, x_2 \ldots, x_n)$: wff

例如: $\{x, y \mid E(x, y, "manager")\}$

机械工业出版社
CHINA MACHINE PRESS

查询处理器

- 查询处理将查询转换为过程操作来访问数据。



- 分布查询处理器还需要处理查询分解（**query decomposition**）和数据局部化（**data localization**）

查询处理器

查询处理问题

- 集中查询处理器必须：
  - ✓ 将演算查询转换为代数操作
  - ✓ 选择最好的执行计划
- 例如：

```
SELECT ENAME
FROM E,G
WHERE RESP = “Manager” and
        E.ENO=G.ENO
```

查询处理器

■关系代数 1: $\pi_{ENAME}(\sigma_{RESP="Manager" \wedge E.EMO=G.ENO}(E \times G))$

■关系代数2: $\pi_{ENAME}(E \infty_{ENO}(\sigma_{RESP="Manager"}(G)))$

■可见： 执行计划 2 好

■对于分布式数据库，查询处理器必须考虑通信代价和选择最佳场地!

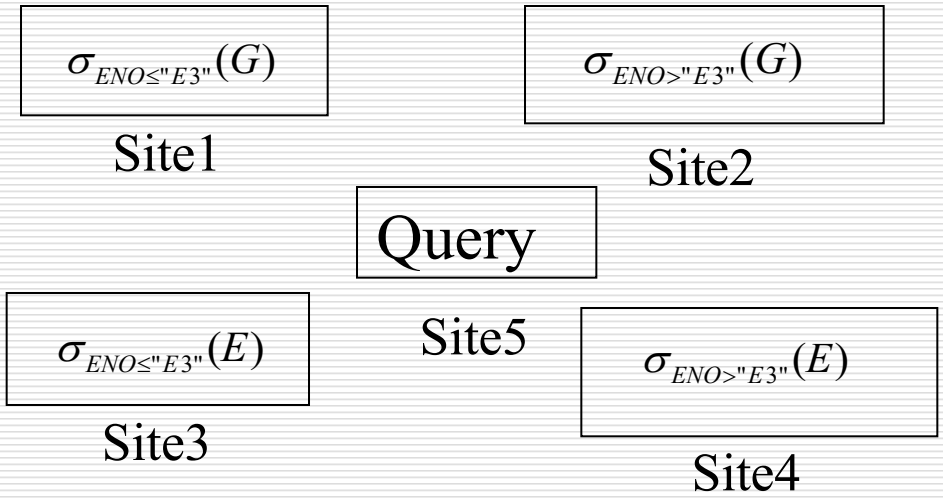
如上例：若 G 和 E 是分布的.

简单计划是将所有片段传到查询场地并执行，但通信代价太大。

机械工业出版社
CHINA MACHINE PRESS

查询处理器

分布式查询举例：



查询：

$$\pi_{ENAME}(E\infty_{ENO}(\sigma_{REPSP="Manager"}(G)))$$

机械工业出版社
CHINA MACHINE PRESS

查询处理器

- 优化的查询

$$G_1' = \sigma_{RESP="Manager"}(G1) \qquad G_2' = \sigma_{RESP="Manager"}(G2)$$

$$E_1' = \pi_{ENAME}(G_1' \infty E1) \qquad E_2' = \pi_{ENAME}(G_2' \infty E2)$$

$$E_1' \cup E_2'$$

查询处理器

- 查询处理的目标
  - 两方面：
    转换（transformation）和优化（optimization）
  - 考虑的优化代价：
    - ✓ CPU time
    - ✓ I/O time
    - ✓ Communication time
  - 在WAN内，侧重通信代价
    在LAN内，三者同等重要

查询处理器

优化类型

- 穷举法（**Exhaustive search**）– 用于小规模解空间
- 启发式（**Heuristics**） – σ，π优先，然后是半连接等，用于大规模解空间

查询处理器

优化时机

- 静态的(Static) – 在编译时进行优化，基于统计信息进行穷举优化，优化一次，执行多次
- 动态的(Dynamic) – 在执行中优化，准确性好，但每次执行都进行优化，代价高

查询处理器

- 统计数据（Statistics）

  元组基数，属性值分布，关系大小等。信息提供给查询优化器并定期修改。

- 决策场地（Decision Site）

查询优化场地可以：

  - 单场地：集中的方法， 简单，需要整个分布库的知识。
  - 分布式：涉及所有的场地协调确定调度，只需本地知识，需要协调代价。
  - 混合型： 一个场地确定全局调度，其他场地优化局部子查询。

- 复制的片段

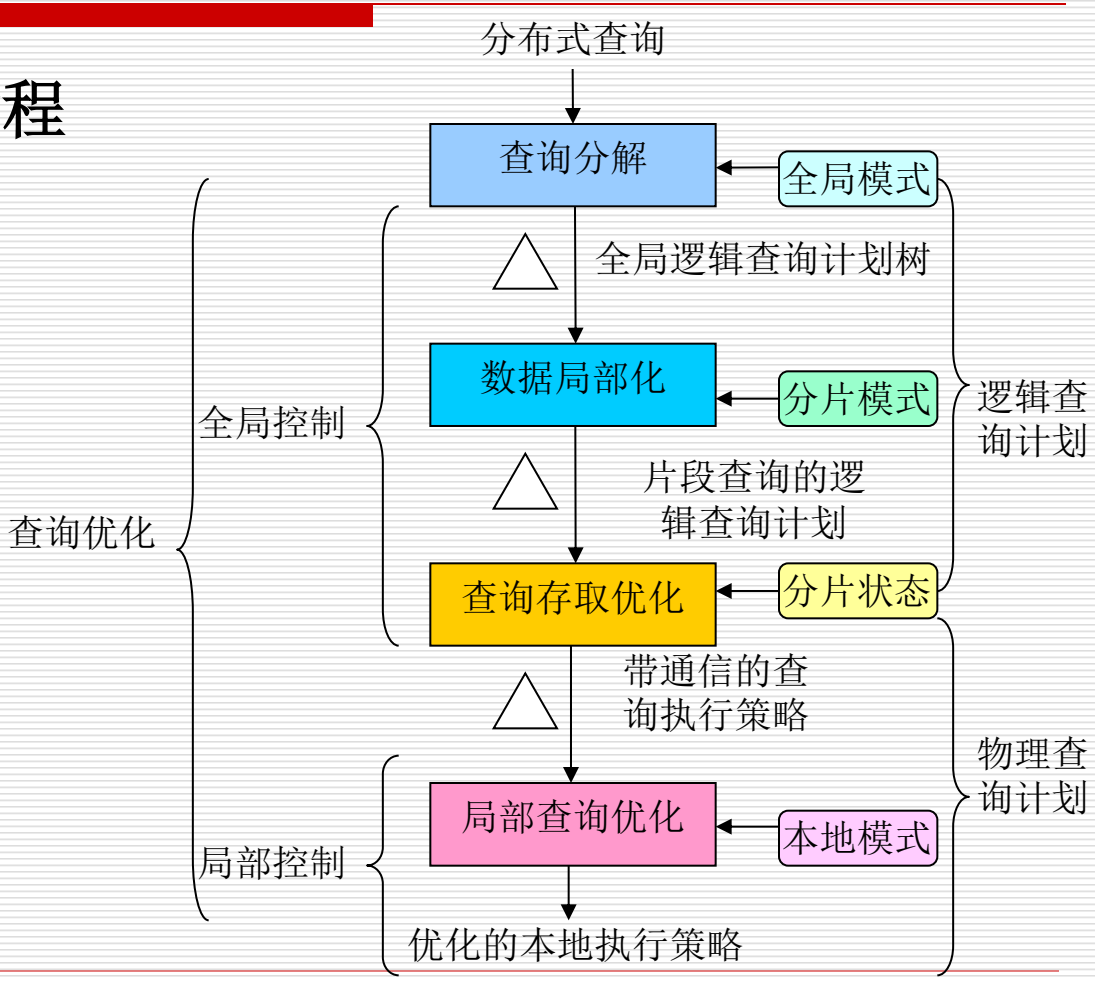
  最小化通信代价。

- 使用半连接

各种连接对比：
https://blog.csdn.net/e2788666/article/details/131259484

查询处理器

- 分布式查询处理过程
  （4个层次）



查询处理器

□ 查询分解（Query Decomposition）

基于全局概念模式将演算查询分解为代数查询。

Step1 – 演算规范化

Step2 – 语义分析，去掉不正确的查询。

Step3 – 简化，去除冗余的部分

Step4 –将演算查询转化为优化的代数查询。

□ 数据局部化（Data Localization）

分布查询映射为片段查询，简化、重组为优化的查询。

查询处理器

查询存取优化（Global Query Optimization）

- 找接近于最优的执行策略；
- 找片段查询中最佳的操作顺序，包括通信操作；
- 需要实时定义代价函数。

局部查询优化（Local Query Optimization）

集中的系统算法.

- INGRES –动态优化（ dynamic optimization）
- System R – 基于穷举法的静态优化（static optimization based on exhaustive search）

查询分解（Decomposition）

- 查询分解（QUERY DECOMPOSITION）

输入：基于全局关系的演算查询

- 规范化（Normalization）
  将查询转换为规范化形式（AND形式或OR形式 ）
- 分析（ Analysis）
  检查不正确的查询
  得到关系演算子集
- 约简（Simplification）
  删除冗余谓词
- 查询重写（Restructuring）
  → 将演算查询转换为关系代数查询
  → 可能得到多种转换结果
  → 使用转换规则

查询分解（Decomposition）

规范化

对于SQL语句，对Where子句中的谓词表示进行规范化。

- ✓词法、语法分析
  - 有效性检查
  - 属性、关系检查
  - 限定条件的类型检查
- ✓ 转换为规范化表示形式
  - 合取范式（Conjunctive normal form）
    $(p_{11} \vee p_{12} \vee \ldots \vee p_{1n}) \wedge \ldots \wedge (p_{m1} \vee p_{m2} \vee \ldots \vee p_{mn})$
  - 析取范式（Disjunctive normal form ）
    $(p_{11} \wedge p_{12} \wedge \ldots \wedge p_{1n}) \vee \ldots \vee (p_{m1} \wedge p_{m2} \wedge \ldots \wedge p_{mn})$
  - OR映射为并操作（union）
  - AND映射为连接（join）或选择（selection）

查询分解（Decomposition）

规范化-转换规则

无量词谓词的转换有如下等价公式：

- 交换律：

$$P_1 \wedge P_2 \Leftrightarrow P_2 \wedge P_1$$

$$P_1 \vee P_2 \Leftrightarrow P_2 \vee P_1$$

- 分配律：

$$P_1 \vee (P_2 \wedge P_3) \Leftrightarrow (P_1 \vee P_2) \wedge (P_1 \vee P_3)$$

$$P_1 \wedge (P_2 \vee P_3) \Leftrightarrow (P_1 \wedge P_2) \vee (P_1 \wedge P_3)$$

- 结合律：

$$P_1 \wedge (P_2 \wedge P_3) \Leftrightarrow (P_1 \wedge P_2) \wedge P_3$$

$$P_1 \vee (P_2 \vee P_3) \Leftrightarrow (P_1 \vee P_2) \vee P_3$$

- 德.摩根定律：

$$\neg(P_1 \wedge P_2) \Leftrightarrow \neg P_1 \vee \neg P_2$$

$$\neg(P_1 \vee P_2) \Leftrightarrow \neg P_1 \wedge \neg P_2$$

- 对合律：

$$\neg(\neg P_1) \Leftrightarrow P_1$$

查询分解（Decomposition）

- 规范化

例: SELECT ENAME
FROM E, G
WHERE E.ENO=G.ENO
AND G.JNO=”J1”
AND (DUR=12 OR DUR=24)

合取范式

E.ENO=G.ENO∧G.JNO=“J1”∧(DUR=12 ∨DUR=24)

析取范式

(E.ENO=G.ENO ∧G.JNO=“J1” ∧ DUR=12) ∨
(E.ENO=G.ENO ∧G.JNO=“J1” ∧ DUR=24)

查询分解（Decomposition）

- 分析

目的: 拒绝类型不正确或语义不正确的查询

✓ *类型不正确*

没定义的关系、属性，错误的类型映射 等

例如:

```
SELECT E#              !undefined attribute
FROM  E
WHERE ENAME>200  !type mismatching
```

查询分解（Decomposition）

- 分析

✓ *语义不正确*

- 查询中存在对查询结果无意义的部分
- 实际上，不可能确定一般查询的语义正确性。但对于不包含 $\vee$ 和 $\neg$ 操作的查询是可以判断的。
- 检查语义不正确的方法

  查询图 (query graph)

  连接图（ join graph）：

  只考虑连接的子图称为连接图（join graph）

查询分解（Decomposition）

■ 分析

✓ *语义不正确*

查询图定义如下：

- 一个点表示结果关系， 其它点表示操作关系
- 边分为两类

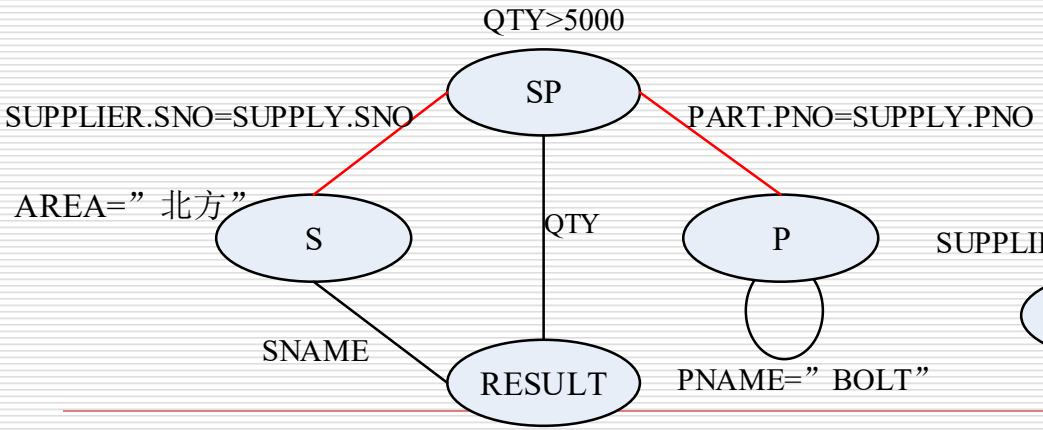
  若边的两点都不是结果，则该类边表示连接(join)

  若边的一个点为结果点，则该边为投影(projection)
- 点和边被标注为选择谓词（predicates for selection）、投影（projection）或连接（ join）

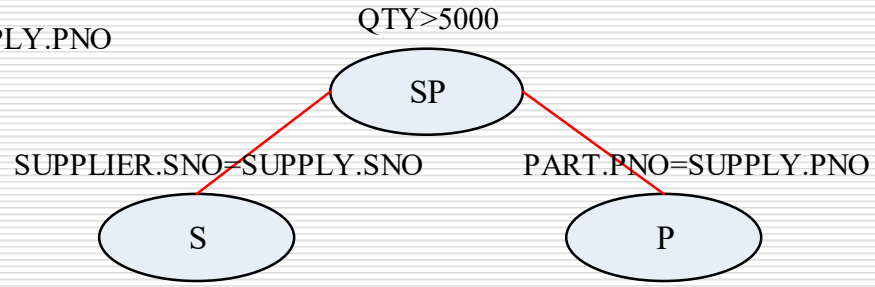
查询分解（Decomposition）

分析-举例

```
SELECT    SNAME，QTY
FROM      SUPPLIER S，SUPPLY SP，PART P
WHERE     S.SNO=SP.SNO
AND       SP.PNO=P.PNO
AND       P.PNAME=“BOLT”
AND       S.AREA=“北方”
AND       SP.QTY>5000;
```



(a) 查询图

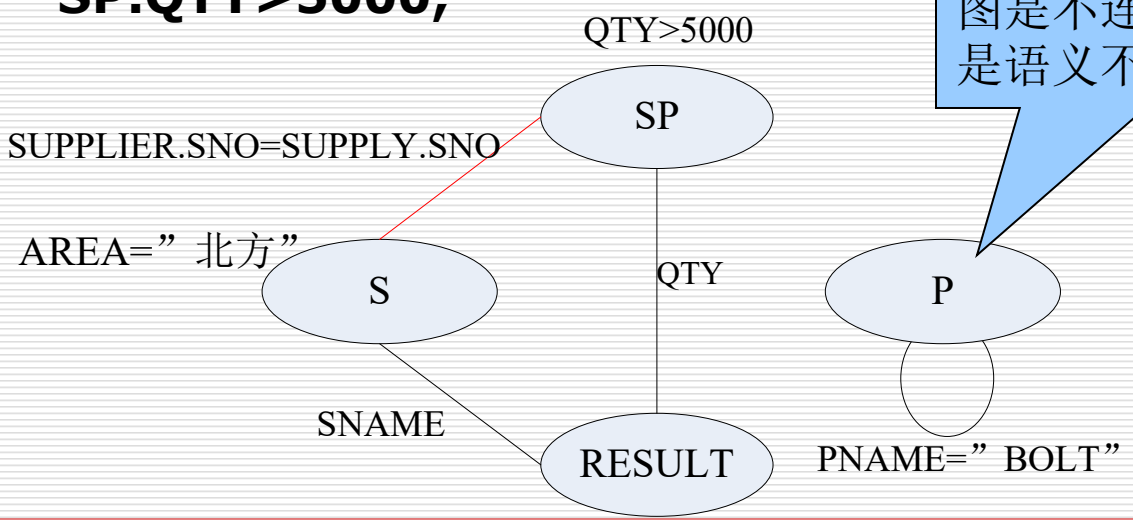


(a) 连接图

查询分解（Decomposition）

分析-举例

```
SELECT    SNAME，QTY
FROM      SUPPLIER S，SUPPLY SP，PART P
WHERE     S.SNO=SP.SNO
AND       P.PNAME=“BOLT”
AND       S.AREA=“北方”
AND       SP.QTY>5000;
```



查询分解（Decomposition）

- 约简

应用转换规则、幂等规则消除冗余

约简规则：

$$P \wedge P \Leftrightarrow P$$

$$P \vee P \Leftrightarrow P$$

$$P \wedge ture \Leftrightarrow P$$

$$P \vee false \Leftrightarrow P$$

$$P \wedge false \Leftrightarrow false$$

$$P \vee ture \Leftrightarrow ture$$

$$P \wedge \neg P \Leftrightarrow false$$

$$P \vee \neg P \Leftrightarrow ture$$

$$P_1 \wedge (P_1 \vee P_2) \Leftrightarrow P_1$$

$$P_1 \vee (P_1 \wedge P_2) \Leftrightarrow P_1$$

查询分解（Decomposition）

- 约简-例如

```
SELECT      PNAME
FROM        PART
WHERE       (NOT (PNAME="BOLT")
AND         (PNAME=" SCREW"
OR          PNAME="BOLT")
AND         NOT (PNAME=" SCREW"))
OR          PNAME=" NUT"
```

⟺

```
SELECT PNAME
FROM   PART
WHERE PNAME="NUT"
```

设P_1表示PNAME= "BOLT"，P_2表示PNAME="SCREW"，P_3表示PNAME= "NUT"，则查询可以表示为：

$$(\neg P_1 \wedge (P_1 \vee P_2) \wedge \neg P_2) \vee P_3 \Leftrightarrow (\neg P_1 \wedge ((P_1 \wedge \neg P_2) \vee (P_2 \wedge \neg P_2))) \vee P_3$$

$$\Leftrightarrow (\neg P_1 \wedge P_1 \wedge \neg P_2) \vee (\neg P_1 \wedge P_2 \wedge \neg P_2) \vee P_3$$

$$\Leftrightarrow (false \wedge \neg P_2) \vee (\neg P_1 \wedge false) \vee P_3 \Leftrightarrow false \vee false \vee P_3 \Leftrightarrow P_3$$

查询分解（Decomposition）

■ 查询重写

- ✓直接将关系演算转换为关系代数；
- ✓重写关系代数查询以提高性能。

■通常使用查询树来表示关系代数查询。

- ✓查询树定义为：
  - 根节点表示查询结果
  - 叶子节点表示关系；
  - 非叶子节点表示中间结果关系
  - 叶子到根的边表示序列操作.

查询分解（Decomposition）

■ 查询重写

✓SQL查询转换为查询树：

- 叶子来自于**FROM**子句；
- 根节点是结果关系，由所需要属性的投影操作生成，包含在**SELECT**子句中；
- **WHERE**子句中的条件被转换成从叶子节点到根节点的关系操作序列，操作序列由操作符和谓词出现的顺序直接生成。

查询分解（Decomposition）

■ 查询重写

查询重写是将用户请求构成的查询树进行等价变换

规则1: 连接、笛卡尔积的交换律

$R \times S \Leftrightarrow S \times R$，$R \infty S \Leftrightarrow S \infty R$

规则2: 连接、笛卡尔积的结合律

$(R \times S) \times T \Leftrightarrow R \times (S \times T)$，$(R \infty S) \infty T \Leftrightarrow R \infty (S \infty T)$

规则3: 投影的串接定律

$\pi_{A1,A2,\ldots,An}(\pi_{B1,B2,\ldots,Bn}(E)) \Leftrightarrow \pi_{A1,A2,\ldots,An}(E)$

规则4: 选择的串接定律

$\sigma_{P1}(\sigma_{P2}(R)) \Leftrightarrow \sigma_{P1 \wedge P2}(R)$

规则5: 选择和投影的交换律

$\sigma_{P}(\pi_{A1,A2,\ldots,An}(R)) \Leftrightarrow \pi_{A1,A2,\ldots,An}(\sigma_{P}(R))$

查询分解（Decomposition）

■ 查询重写

规则6: 选择与笛卡尔积的分配律

$$\sigma_P(R \times S) \equiv \sigma_P(R) \times S$$

规则7: 选择与并的分配律

$$\sigma_P(R \cup T) \Leftrightarrow \sigma_P(R) \cup \sigma_P(T)$$

规则8: 选择与差的分配律

$$\sigma_P(R-T) \Leftrightarrow \sigma_P(R)-\sigma_P(T)$$

规则9: 投影与笛卡尔积的分配律

$$\pi_{A1,A2,\ldots,An,\ B1,B2,\ldots,Bn}(R \times S) \equiv \pi_{A1,A2,\ldots,An}(R) \times \pi_{B1,B2,\ldots,Bn}(S)$$

规则10: 投影与并的分配律

$$\pi_{A1,A2,\ldots,An}\ (R \cup F) \equiv \pi_{A1,A2,\ldots,An}(R) \cup \pi_{A1,A2,\ldots,An}(F)$$

查询分解（Decomposition）

■ 查询重写

✓ 应用等价变换规则，一棵查询树可等价转换为多棵查询树，需要能生成最优查询树的等价变换。

✓ 关系查询优化的基本思想是先做能使中间结果变小的操作，尽量减少查询执行代价。

运算类型	执行代价
σ、Π（不消重复项）	O（n）
Π（消重复项）、GROUP	O（nLOGn）
×	O（n^2）
∞、∪、∩、-、∝、÷	O（nLOGn）

变换的通用准则为：

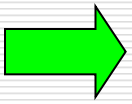
准则1 尽可能将一元运算移到查询树的底部（树叶部分），优先执行一元运算。

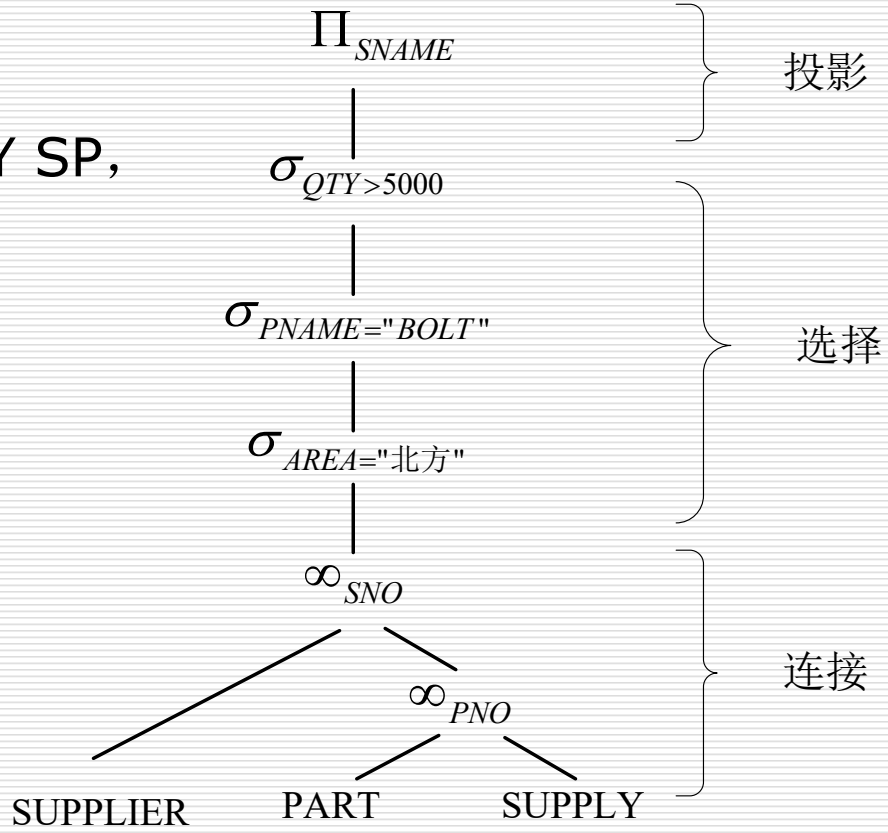
准则2 利用一元运算重复律，缩减每一关系，减少关系尺寸，降低网络传输量和I/O大小。

查询分解（Decomposition）

查询重写

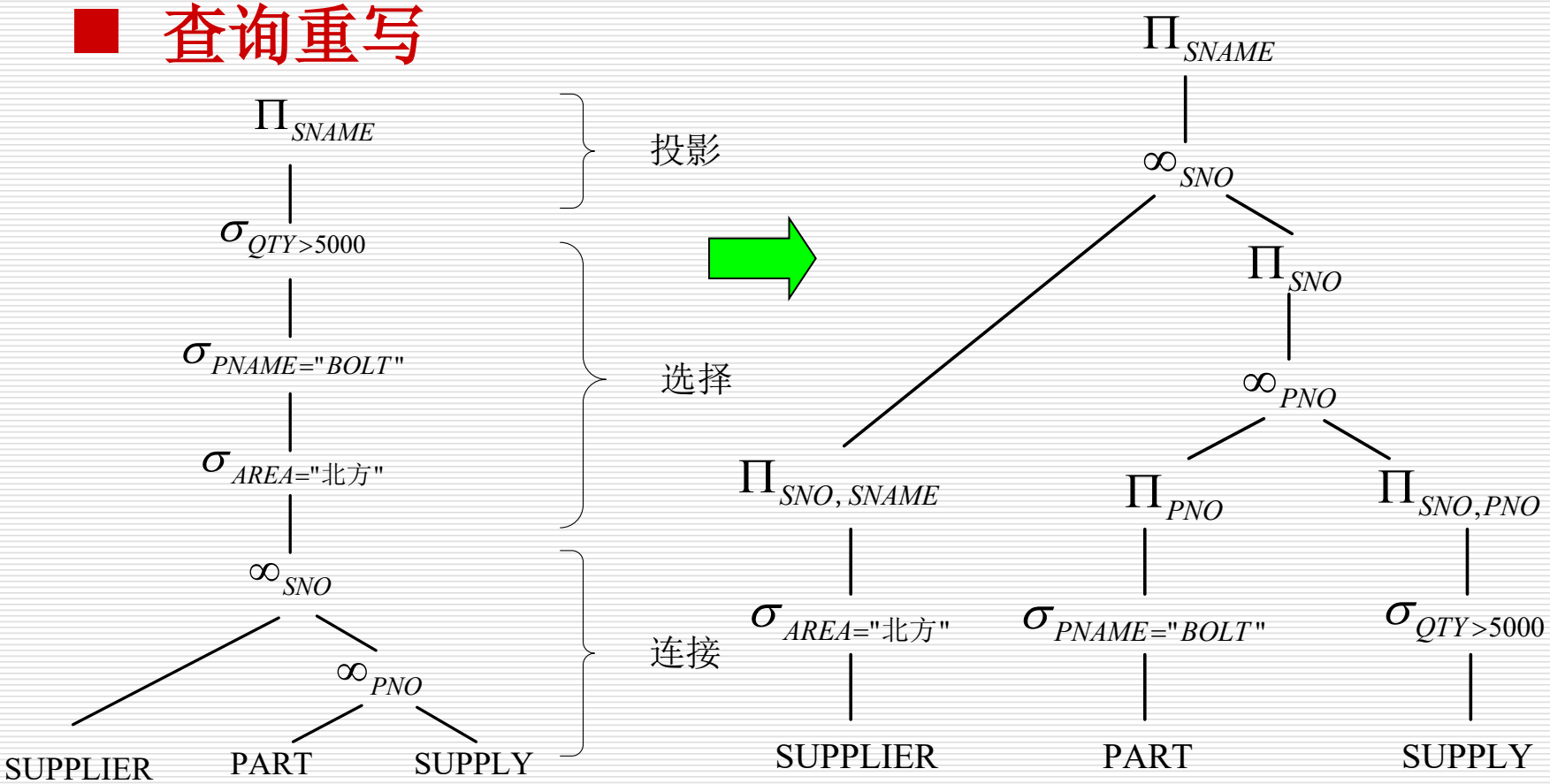
```
SELECT SNAME
FROM  SUPPLIER S，SUPPLY SP，
PART P
WHERE S.SNO=SP.SNO
AND SP.PNO=P.PNO
AND P.PNAME=“BOLT”
AND S.AREA=“北方”
AND SP.QTY>5000;
```





查询分解（Decomposition）

查询重写



查询分解（Decomposition）

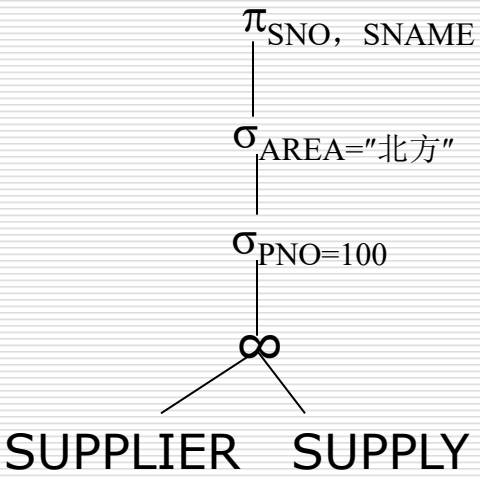
- 举例

SUPPLIER{SNO，SNAME，AREA}
SUPPLY{SNO，PNO，QTY}

等价的关系表达式：

Q1： $\pi_{SNO,SNAME}(\sigma_{AREA="北方"}(\sigma_{PNO=100}(SUPPLIER \infty SUPPLY)))$

查询树：

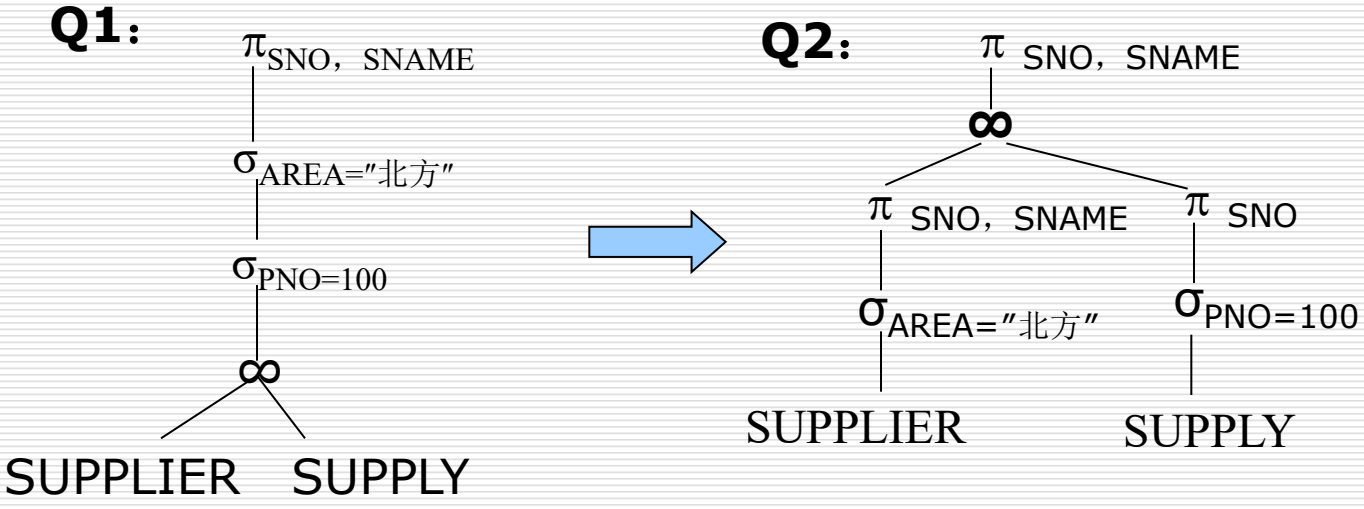


查询分解（Decomposition）

- 全局优化

基于查询重写准则（在查询树**Q1**的基础上进行全局优化）

根据分配律，将一元运算向下移。得到全局优化后的查询树：



数据局部化

- 查询的处理过程是从全局关系到片段关系，最后再到实际操作的副本关系。
- 数据本地化将介绍全局查询到片段查询的变换。即利用全局关系与其片段关系的等价变换，将分布查询中的全局关系替换为对片段关系的查询，变换后的查询称为片段查询。对应于片段查询的查询树，称为片段查询树。

数据局部化

分布查询与片段查询的等价关系

- 对于全局关系R的水平分片R_1，R_2，…，R_n，表示为：
  $R=R_1 \cup R_2 \cup \ldots \cup R_n$
- 对于全局关系R的垂直分片R_1，R_2，…，R_n，表示为：
  $R=R_1 \infty R_2 \infty \ldots \infty R_n$

片段查询树的生成步骤

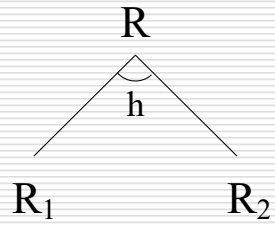
- 将分片树的h（水平）节点转换为查询树的∪（并集）节点。
- 将分片树的v（垂直）节点转换为查询树的∞（连接）节点。
- 用替换后的分片树代替全局查询树中的全局关系，得到片段查询树。

数据局部化

片段查询树的生成步骤

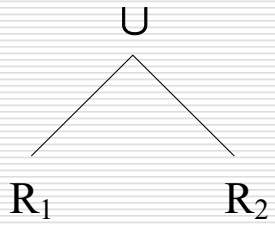
具体如下：

水平h节点分片树

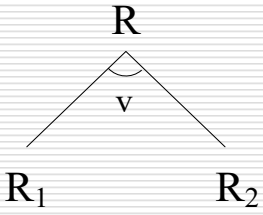


===》

转换后的∪节点

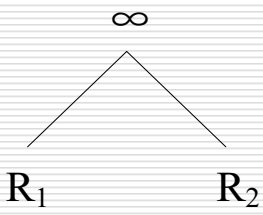


垂直v节点分片树



===》

转换后的∞节点



数据局部化

- **例：** 假设SUPPLIER和SUPPLY的分片如下：
  - SUPPLIER水平分片为S_1和S_2，具体如下：

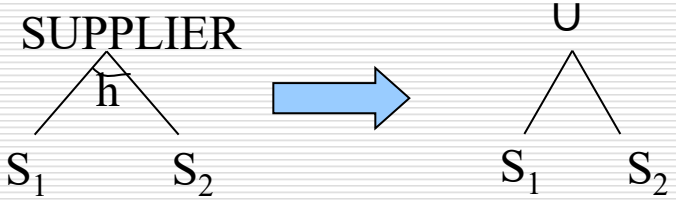
    $S_1=\sigma_{AREA=\text{"北方"}}$（SUPPLIER）

    $S_2=\sigma_{AREA=\text{"南方"}}$（SUPPLIER）
  - SUPPLY水平分片为$SUPPLY_1$和$SUPPLY_2$，具体如下：

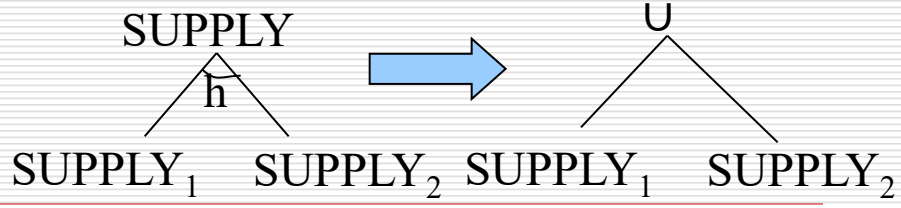
    $SUPPLY_1=\sigma_{AREA=\text{"北方"}}$（SUPPLY）

    $SUPPLY_2=\sigma_{AREA=\text{"南方"}}$（SUPPLY）

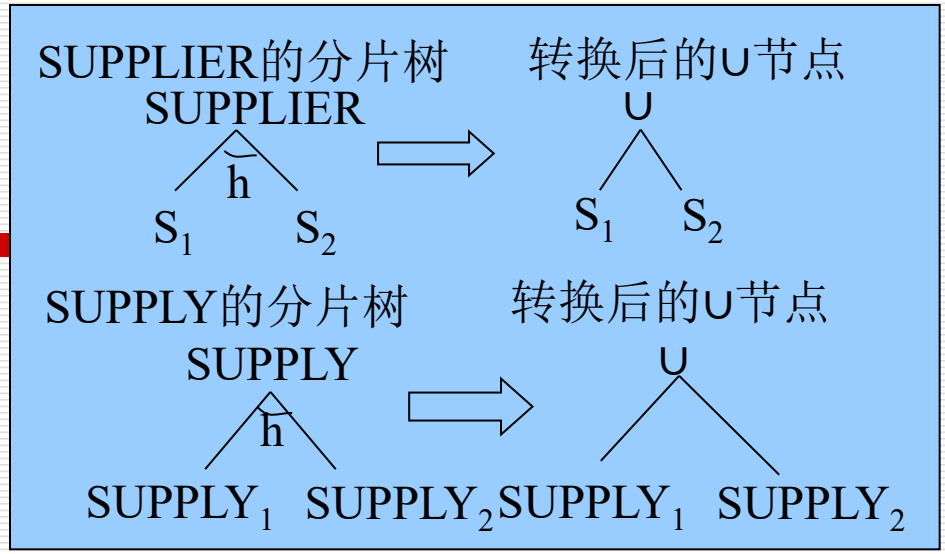
SUPPLIER的分片树为：　　转换后的∪节点：



SUPPLY的分片树为：　　转换后的∪节点：



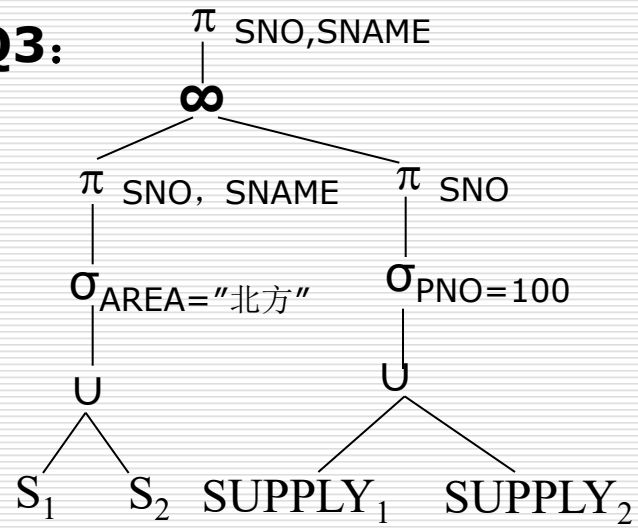
数据局部化



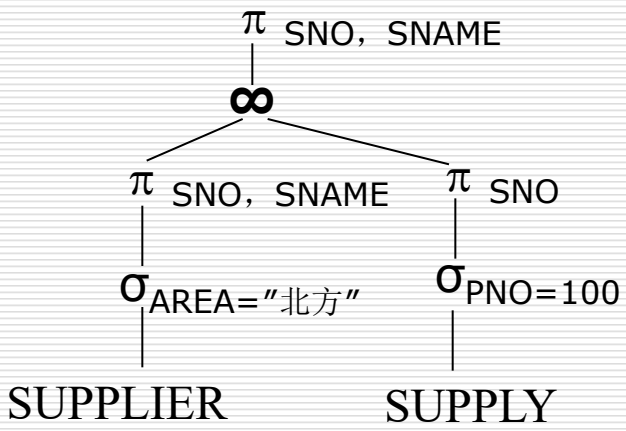
■ 转换为片段查询树Q3

在Q2基础上，用SUPPLIER分片树替换查询树Q2的全局关系SUPPLIER，用SUPPLY分片树替换查询树Q2的全局关系SUPPLY，即得到转换后的片段查询树为Q3。

Q3：



Q2：



片段查询的优化

□ 片段查询优化规则

准则1：对于一元运算，根据一元运算的重复律，将叶子节点之前的选择运算作用于片段，如果不满足片段的限定条件，则置为空关系。

准则2：对于连接运算的树，若连接条件不满足，则将其置为空关系。

准则3：在查询树中，将连接运算（∞）下移到并运算（U）之前执行。

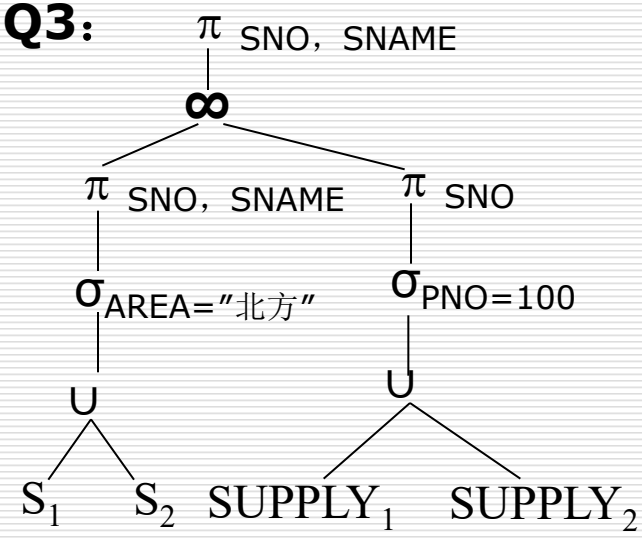
准则4：消去不影响查询运算的垂直片段。

片段查询的优化

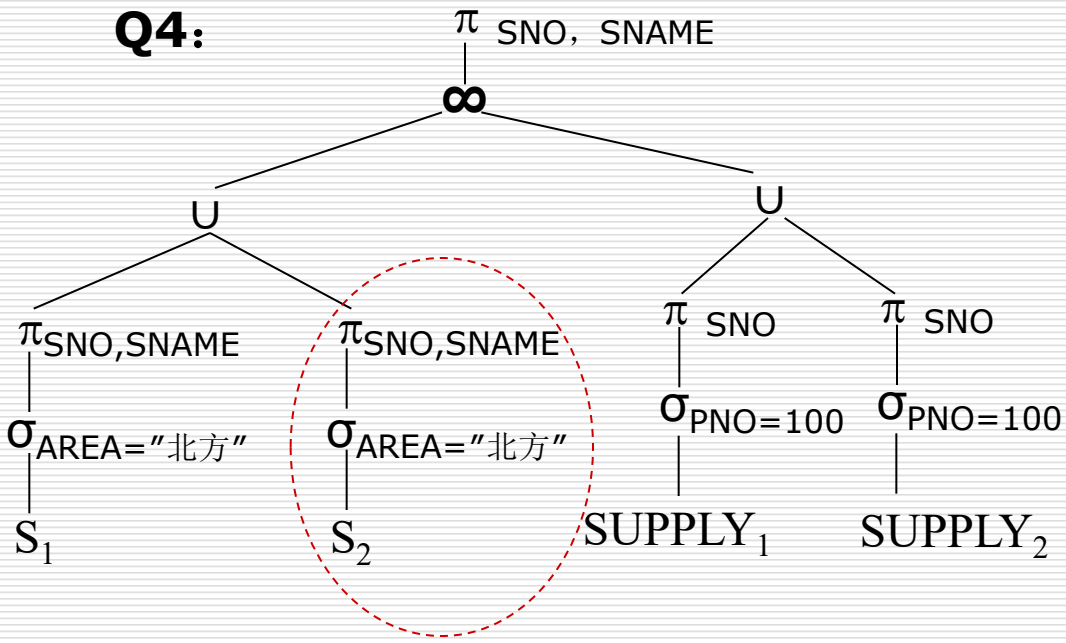
- 接上例，以片段查询树**Q3**为基础，

（**1**）根据片段查询优化准则**1**，按限定条件化简，得到**Q4**。

Q3：



Q4：



片段查询的优化

（2）根据

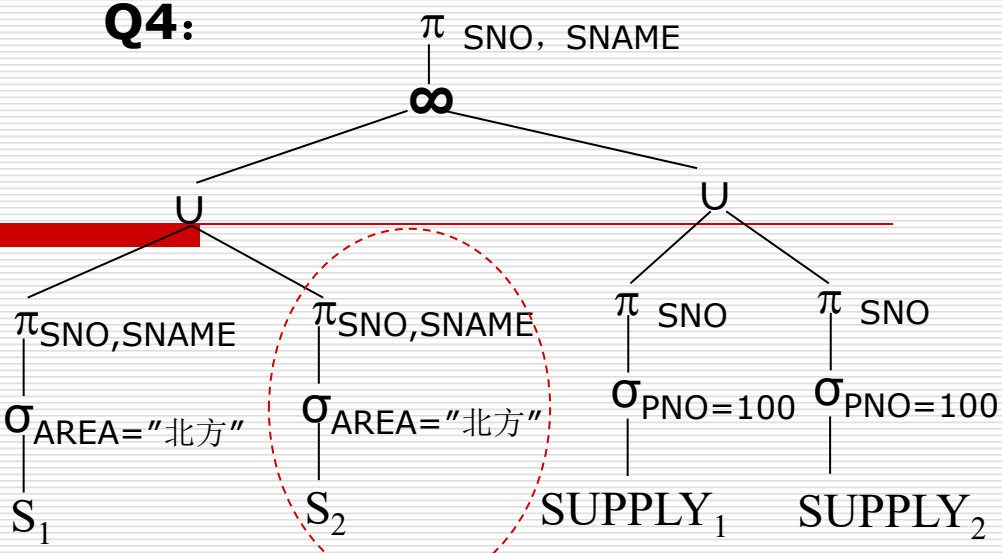
$S_1=\sigma_{AREA="北方"}(SUPPLIER)$

$S_2=\sigma_{AREA="南方"}(SUPPLIER)$

$SUPPLY_1=\sigma_{AREA="北方"}(SUPPLY)$

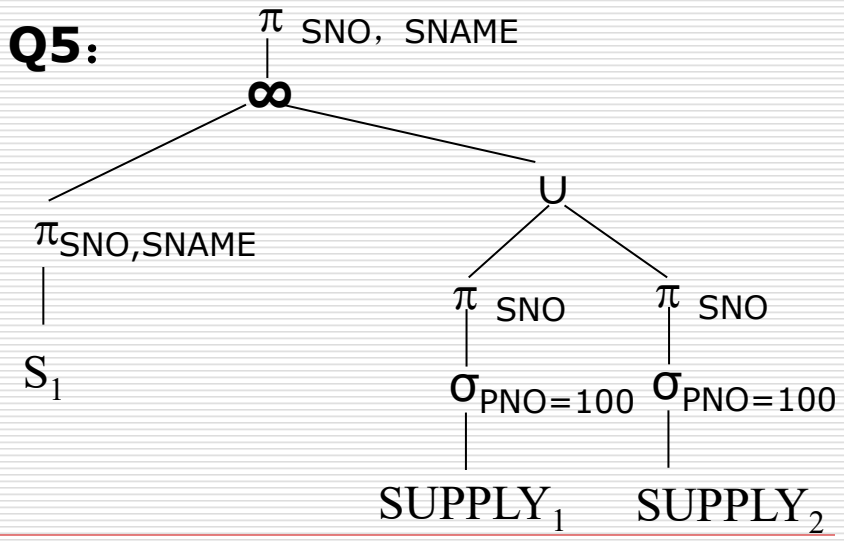
$SUPPLY_2=\sigma_{AREA="南方"}(SUPPLY)$

Q4：



所以，Q4中的虚线部分为空关系，可将其去掉，得到Q5查询树。

Q5：

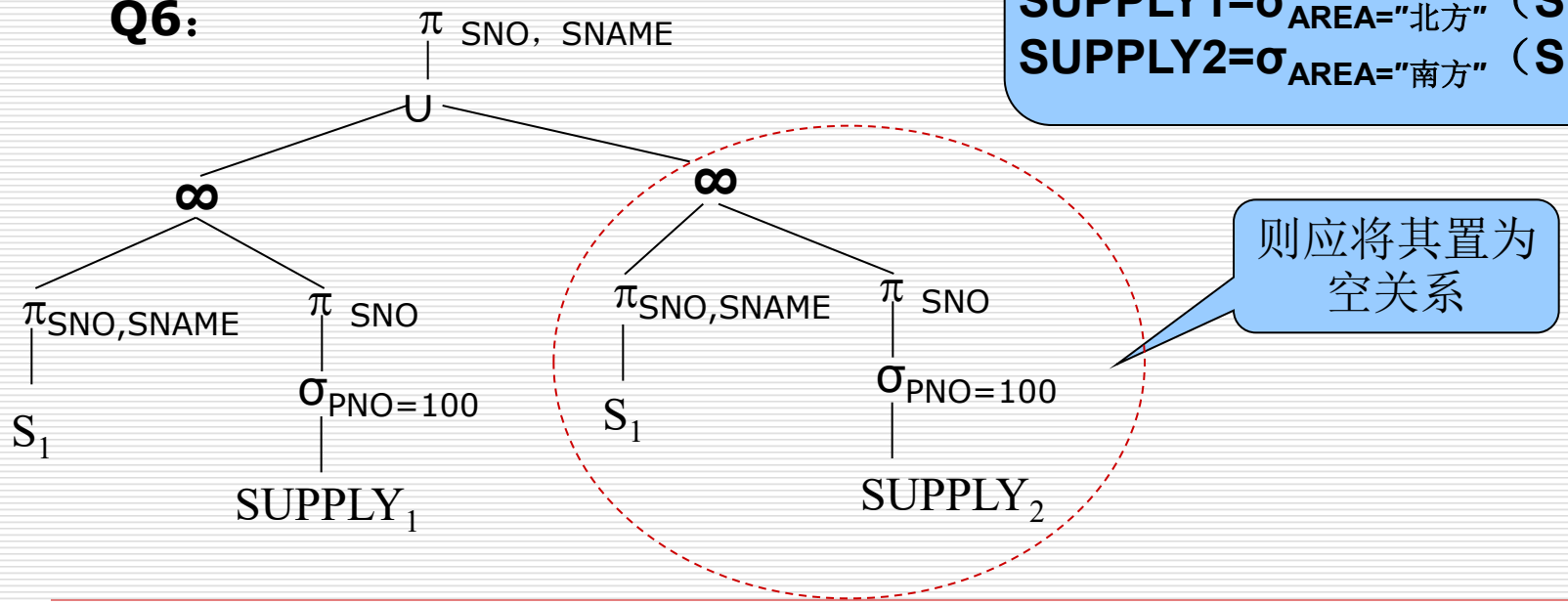


片段查询的优化

（3）根据准则3，将连接运算(∞)下移到并运算(∪)之前，得到Q6。

$S1=\sigma_{AREA="北方"}(SUPPLIER)$
$S2=\sigma_{AREA="南方"}(SUPPLIER)$
$SUPPLY1=\sigma_{AREA="北方"}(SUPPLY)$
$SUPPLY2=\sigma_{AREA="南方"}(SUPPLY)$

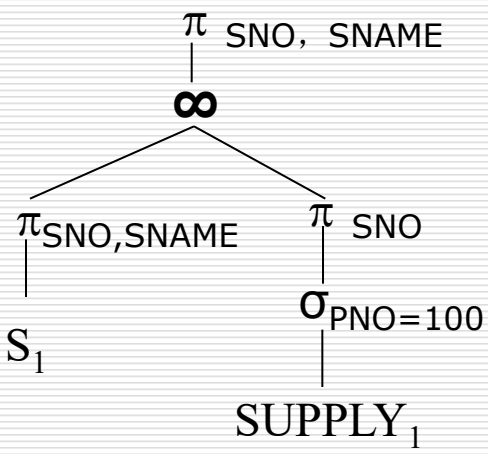
Q6：



片段查询的优化

得到**Q7**：

Q7：



Q7为化简后的最终的优化查询。

片段查询的优化

- 例
  - 假设：一雇员关系 EMP {ENO，ENAME，BIRTH，SALARY，DNO}
    - ✓ 其分片为：

$E1=\Pi_{ENO，ENAME，BIRTH}（EMP）$

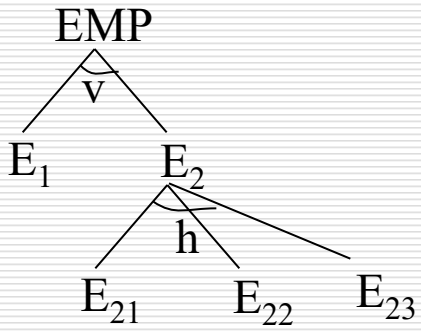
$E2=\Pi_{ENO，SALARY，DNO}（EMP）$

$E21=\sigma_{Dno=201}（E2）$

$E22=\sigma_{Dno=202}（E2）$

$E23=\sigma_{Dno〈〉201\ AND\ Dno〈〉202}（E2）$

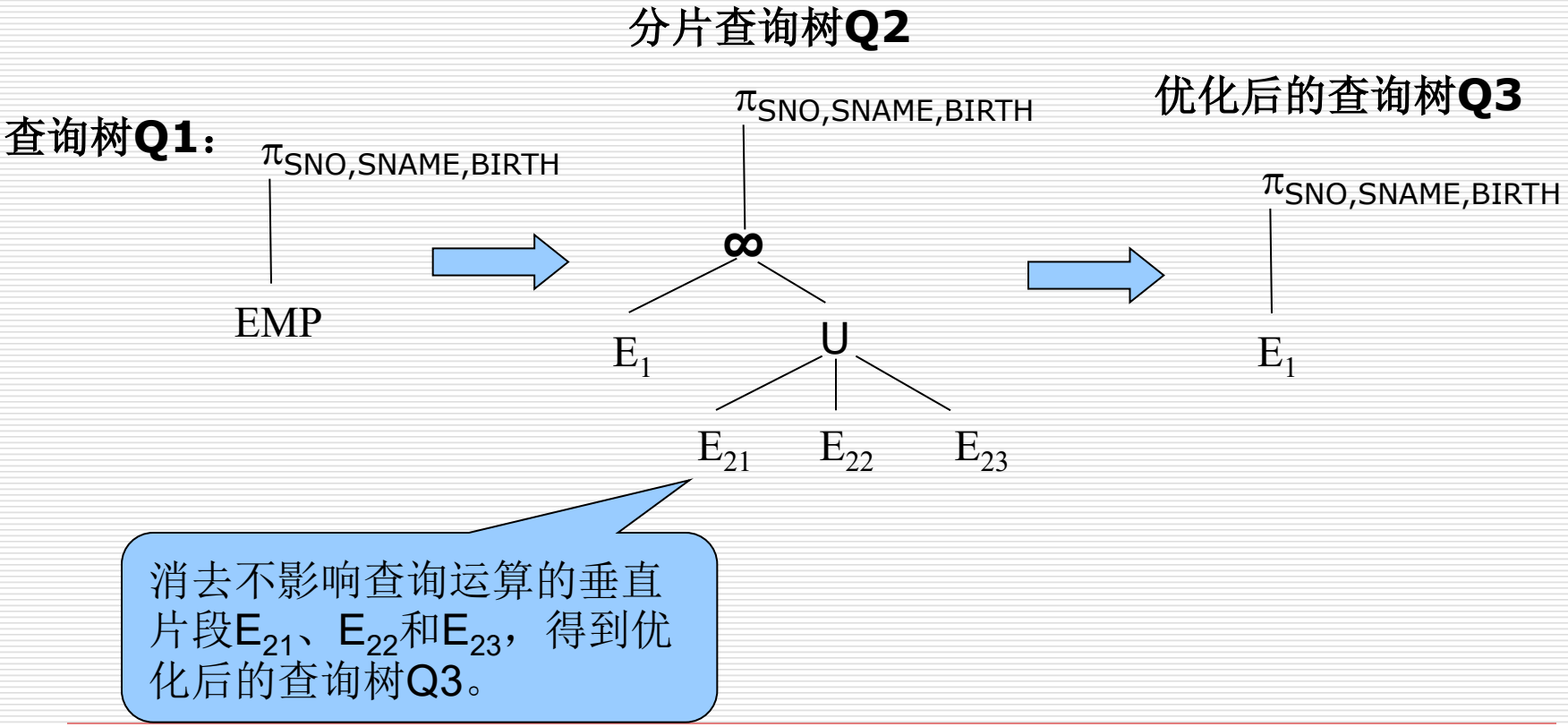
- ✓ 分片树：



要求：查询SQL：SELECT ENO，ENAME，BIRTH FROM EMP

片段查询的优化

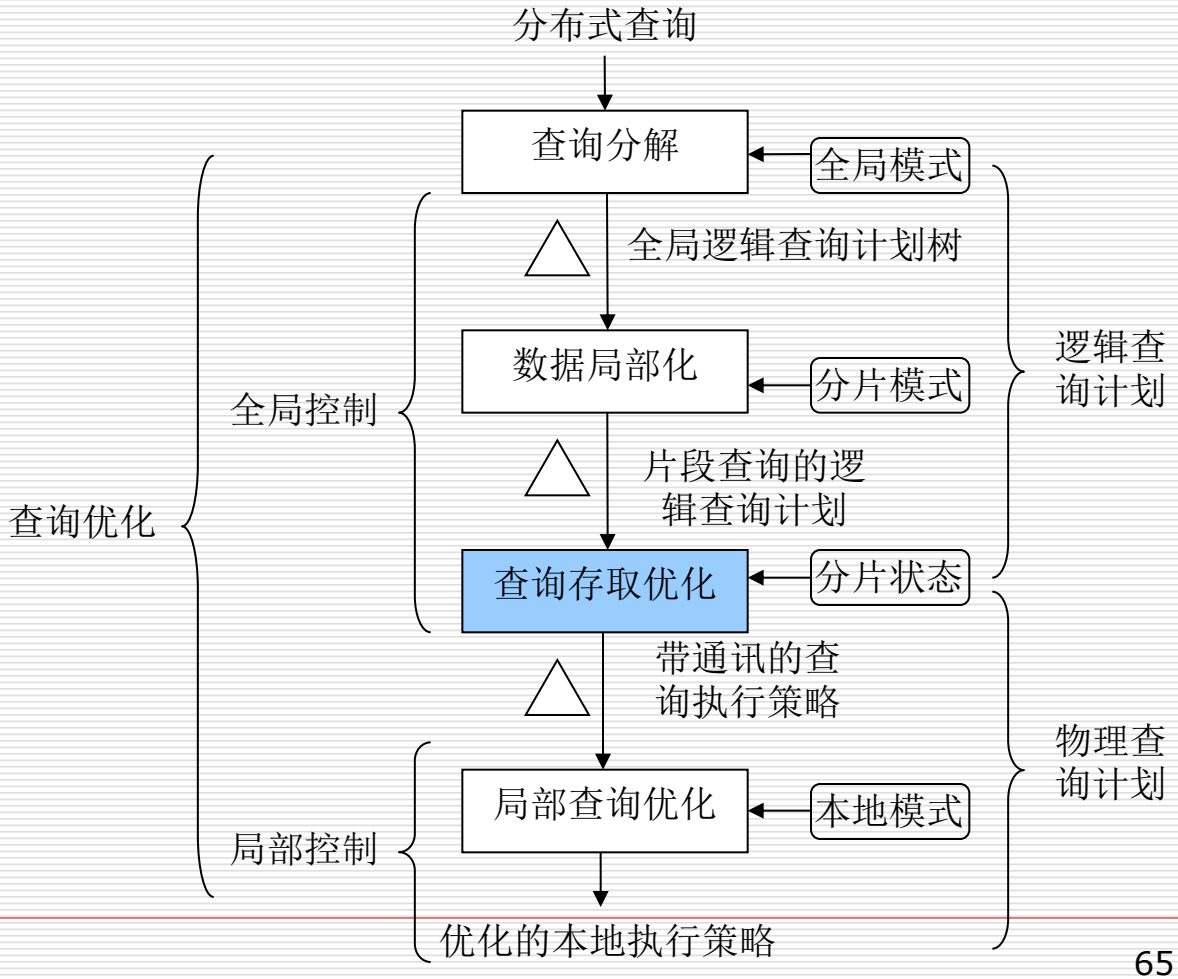
- 例


查询树Q1：
πSNO,SNAME,BIRTH
EMP
分片查询树Q2
πSNO,SNAME,BIRTH
∞
E1
U
E21
E22
E23
优化后的查询树Q3
πSNO,SNAME,BIRTH
E1
消去不影响查询运算的垂直片段E21、E22和E23，得到优化后的查询树Q3。

2.分布式查询的存取优化

- 查询存取优化概述
- 基本概念
- 存取优化的理论基础
- 半连接优化方法
- 枚举法优化技术

查询存取优化概述


分布式查询
查询分解
全局模式
全局逻辑查询计划树
数据局部化
分片模式
片段查询的逻辑查询计划
查询存取优化
分片状态
带通讯的查询执行策略
局部查询优化
本地模式
优化的本地执行策略
全局控制
局部控制
查询优化
逻辑查询计划
物理查询计划

查询存取优化概述

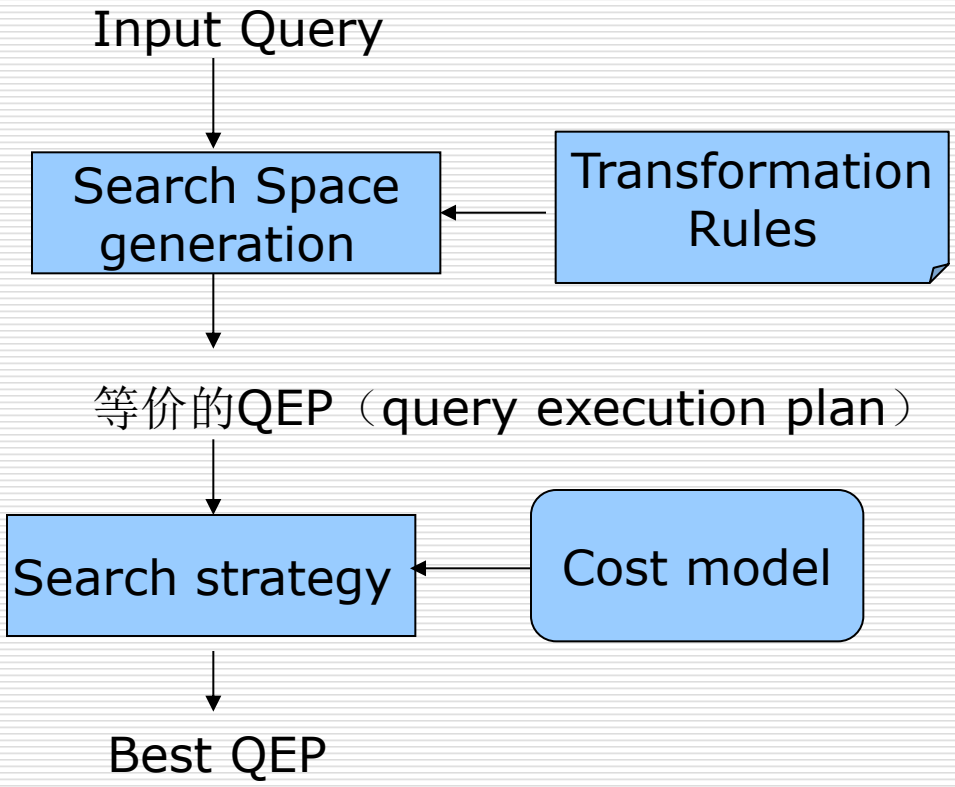
- 查询存取优化
  - 输入：片段查询
  - 找到好的（不必最优的）全局调度
    - ✓最小代价函数
    - ✓分布Join处理
      - ●稠密树与线性树
      - ●需要传输的关系
      - ●全部传输与按需传输
    - ✓确定使用半连接
      - ●使用半连接可减少通信代价
    - ✓Join方法
      - ●嵌套循环与顺序join（合并join与hash join）

查询存取优化概述

- 基于代价的优化
  - 搜索空间(Solution space)
    - ✓等价的代数表达式 (query trees)
  - 代价函数(Cost function)
    - ✓I/O 代价 + CPU 代价+ communication代价
    - ✓不同的分布环境中各因素权重不同 (LAN vs WAN)
    - ✓最大吞吐量
  - 搜索算法(Search algorithm)
    - ✓如何在解空间中移动
    - ✓穷举搜索，启发式算法 (递归改进、模拟退火、遗传法等)

查询存取优化概述

查询优化过程



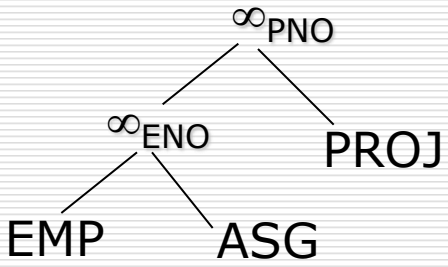
- 查询优化
  - ✓ 最小化查询代价

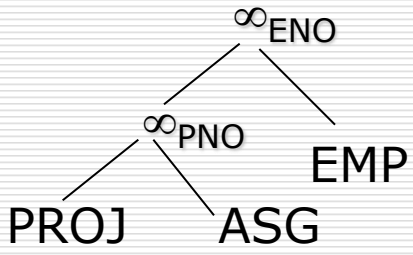
查询存取优化概述

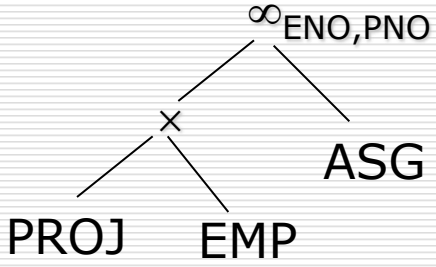
- 搜索空间(**Search Space**)
  - 搜索空间由可选择的执行计划描述
  - 侧重Join树
  - 若存在n个关系，有O(N!)等价的Join树

例如，

```
select ENAME,RESP
From EMP,ASG,PROJ
Where EMP.ENO=ASG.ENO
And ASG.PNO=PROJ.PNO
```



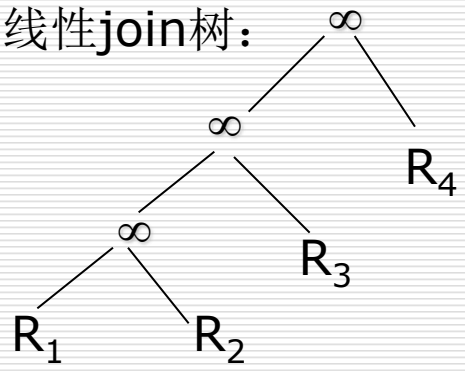




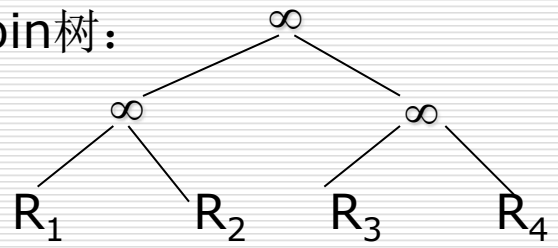
查询存取优化概述

- 搜索空间(Search Space)
  - 由启发式规则约束
    如一元操作先于二元操作执行
  - 约束join树的形状
    只考虑线性树而忽略稠密树

线性join树：



稠密join树：



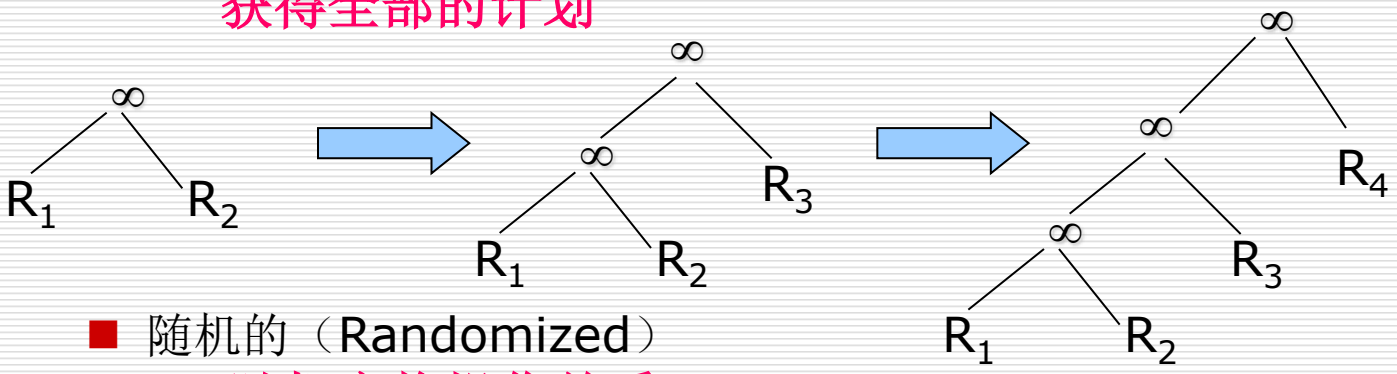
查询存取优化概述

- 搜索策略(**Search Strategy**)
  - 如何在搜索空间中构建查询计划
  - 确定的（Deterministic）
    - ✓开始于基关系，每步加一个关系构建执行计划
    - ✓动态规划法：宽度优先（breadth-first）
    - ✓贪婪法：深度优先（depth-first）（只构造一个计划）
  - 随机的（Randomized）
    - ✓ 从一个特定的点起始搜索最优执行计划 (首先应用贪婪法构建一个或多个执行计划，接着通过邻居关系替换来改进执行计划)
    - ✓ 综合考虑优化时间和执行时间
    - ✓ 适合大于 5-6 个关系
    - ✓ 模拟退火（Simulated annealing）
    - ✓ 递归改进（Iterative improvement）

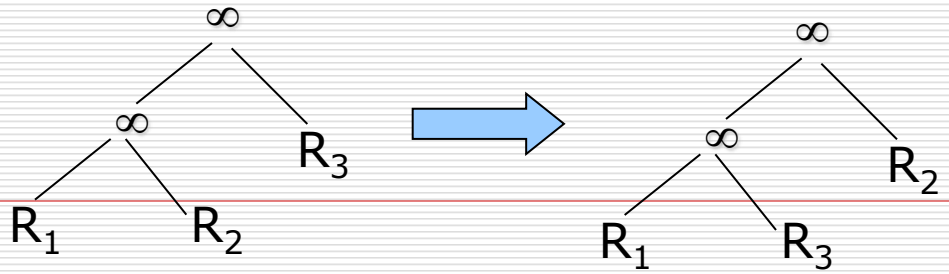
✓构造所有可能的执行计划；
✓尽早剪裁不是优化的计划；
✓Is also exhaustive；
✓关系数少时，优化代价可接受；

查询存取优化概述

- 搜索策略(**Search Strategy**)
  - 确定的（Deterministic）
    - ✓从基关系开始，每步连接一个或多个关系，直到获得全部的计划



  - 随机的（Randomized）
    - ✓随机交换操作关系



机械工业出版社
CHINA MACHINE PRESS

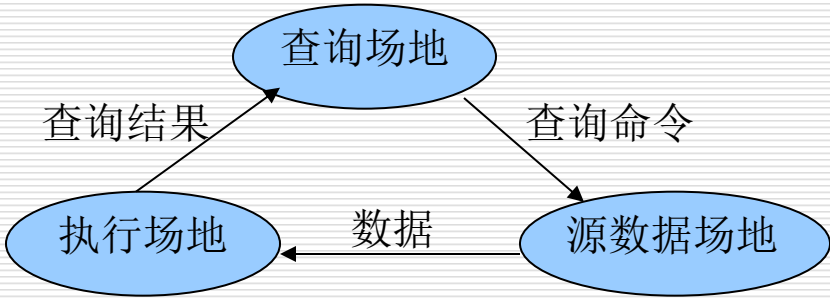
基本概念

- 分布执行过程实际上就是从查询场地发出查询命令、从数据源获取数据、确定最佳的执行场地和返回执行结果的过程。

■查询场地：指发出查询命令和存储最终查询结果的场地。查询场地也称最终结果文件。

■源数据场地：指查询命令需要访问的数据副本所在的场地，可能涉及到一个或一个以上的场地。源数据场地也称源数据文件。

■执行场地：指查询操作执行所在的场地。执行场地可以和查询场地或源数据场地处于同一场地，也可不处于同一场地。执行场地也称中间结果文件。



基本概念

分布执行策略举例

- 有关系EMP和DEPT。
  - ✓ EMP{ENO，ENAME，BIRTH，SALARY，DNO}（ENO为主键）

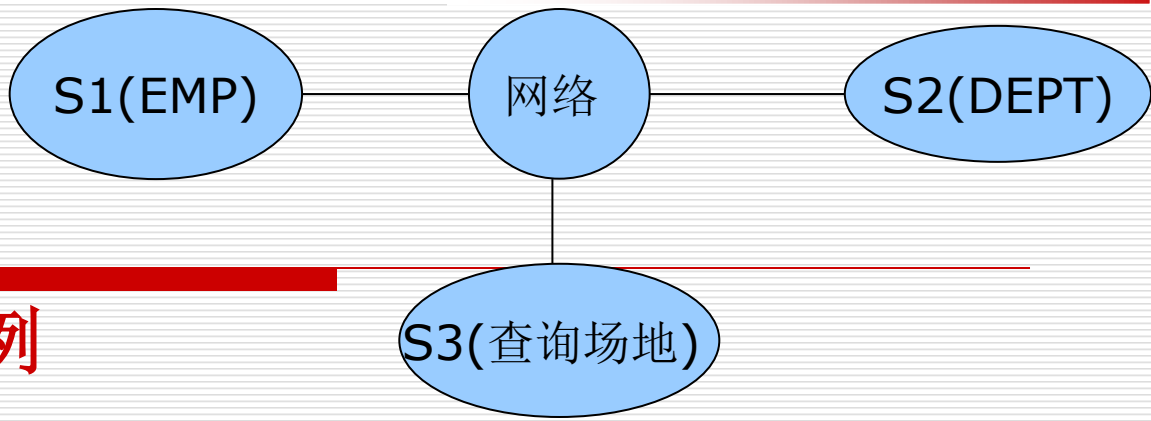
    ENO，ENAME，BIRTH，SALARY，DNO分别为雇员编号 雇员姓名 出生日期 工资 部门号
  - ✓ DEPT{DNO，DNAME}（ENO为主键）

    DNO，DNAME分别为部门号，部门名称
- 假设

（1）EMP：元组数：10000，元组大小：100B，关系大小：100*10000=1000KB

（2）DEPT：元组数：100，元组大小：35B，关系大小：35*100=3.5KB

机械工业出版社
CHINA MACHINE PRESS



基本概念

□分布执行策略举例

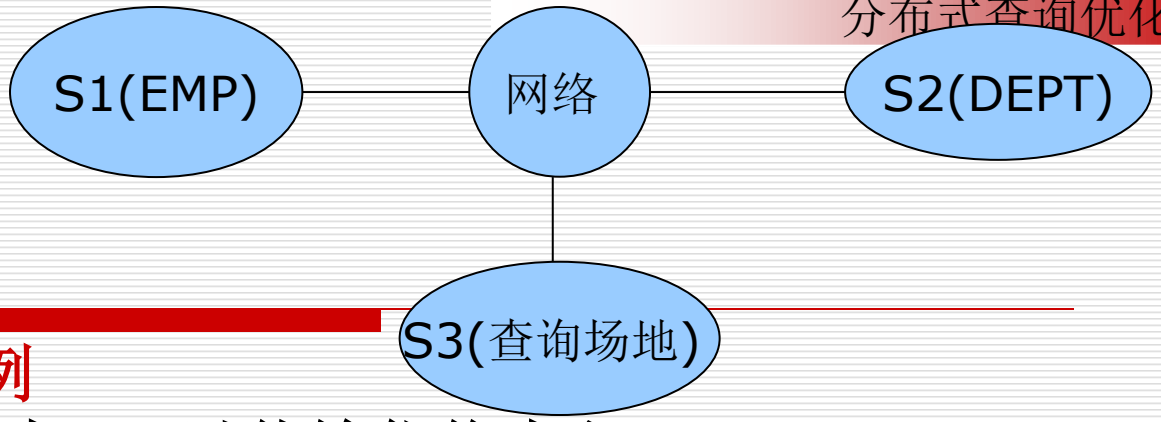
(3) 存在三个场地，S1、S2和S3。如图：

查询要求：查询每个雇员的姓名及所在单位。

则：（1）SQL语句：**SELECT ENAME，DNAME**
FROM EMP，DEPT
WHERE EMP.DNO=DEPT.DNO

（2）关系表达式：$\pi_{ENAME,DNAME}$（EMP∞DEPT）

假设：结果元组大小40字节，S3为查询场地
结果关系大小：40*10000=400KB



基本概念

- 分布执行策略举例
  - 策略（设结果为R，以传输代价为主）

    策略1：

    S3为执行场地，则需传输EMP、DEPT

    传输量=1000KB+3.5KB=1003.5KB

    策略2：

    S2为执行场地，则需传输EMP到S2，结果R传输到S3。

    传输量=1000KB+400KB=1400KB

    策略3：

    S1为执行场地，则需传输DEPT到S1，结果R传输到S3。

    传输量=3.5KB +400KB=403.5KB
  - 从上面三个策略看，选择不同的执行场地，传输代价差别很大。应选择最低的传输代价。但组成系统的环境不同，优化的侧重点也不同。

基本概念

分布查询的存取优化的目标

- 远程网：主要考虑通信开销，使通信代价最小。
- 局域网：需同时考虑通信代价和本地处理代价，使综合代价最小。

优化的内容

优化是在片段查询的基础上进行的实际物理副本查询操作的优化。具体如下：

输入：片段查询表达式

输出：分布执行计划

基本概念

□查询存取优化

内容：

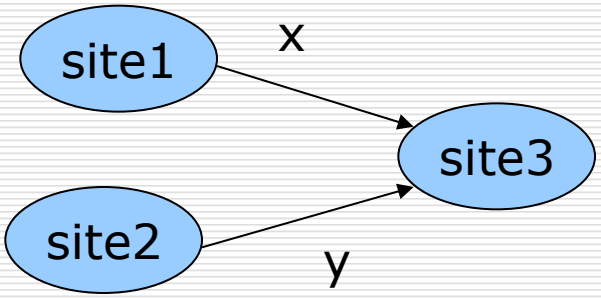
- 确定片段查询需访问的物理副本。通常：①本场地上的物理副本优先；②若二元运算存在尽量选择本场地上的二元运算；③数据最小的物理关系应被优先选中；④网络通信代价小的应优先选中。
- 确定片段查询表达式操作执行的最优顺序。包括从叶到根的执行和同一层叶子上表达式执行的先后，特别是对查询树上的并操作和连接操作的执行次序的确定，其代价差别很大。
- 选择执行每个操作的方法。如：尽量将同一场地上的、同一物理副本的全部操作组合在一起统一考虑完成。

存取优化的理论基础

- 代价函数(the total time or the response time)
  - 总的时间（Total Time）-所有时间组件的和
    - ✓减少每一个组件的时间代价
    - ✓尽可能减少每个代价组件的代价
    - ✓优化资源利用率，增加系统吞吐率
  - 响应时间（Response Time）-从查询开始到执行结束所用的时间
    - ✓尽可能并行执行（Do as many things as possible in parallel）
    - ✓增加总的活动（activity）可能会增加总时间

存取优化的理论基础

- 例子:



- 假设只考虑通信代价
  - ✓总时间=2*消息启动时间+单位传输代价*(x+y)
  - ✓响应时间=max(从1传输x到3的时间，从2传输y到3的时间)

存取优化的理论基础

代价模型

主要指传输代价（C_{com}）、I/O代价（C_{IO}）和CPU代价（C_{cpu}）

Total cost = $C_{com}+C_{IO}+C_{cpu}$

传输代价

费用和延迟。其中，费用起决定作用。

传输费用是指通信中的整个传输开销，即传输的数据量。

模型为：$C_{COM}（X）=C_0+C_1*X$

其中：C_0：场地间传输数据的启动所需的固定费用（启动一次），简称启动代价；

C_1：网络单位传输数据费用，简称单位传输代价；

X：需传输的数据量。

机械工业出版社
CHINA MACHINE PRESS

存取优化的理论基础

- **I/O代价**

模型为：$C_{IO}(X)=[X/P]*C_{IO}$

其中：P：页面的大小；C_{IO}：为每页平均访问代价；

X：数据量大小。

- **CPU代价**

模型：$C_{CPU}(X)=X*C_{CPU}$

其中：C_{CPU}：单位指令代价；X：为指令数。

通常具有下面的统计值：

广域网环境：$C_{COM}/C_{IO}=20:1$；

局域网环境：$C_{COM}/C_{IO}=1.6:1$。

可见，在广域网环境，以传输代价为主；在局域网环境，需综合考虑传输代价和局部代价。

存取优化的理论基础

查询模型

主要代价因素：中间关系大小

- 数据库特征参数

假设**R**为一关系

- ✓ 关系的基：指关系**R**包含的元组个数，记为**Card(R)**
- ✓ 属性的长度：指属性**A**定义的取值字节数，记为**Length(A)**
- ✓ 元组的长度：关系**R**中每个元组的字节数，记为**Length(R)**，**Length(R)**=Σ**Length(**A_i**)**
- ✓ 关系的大小：关系R所包含的字节数，记为**Size(R)**

  Size(R)= Card(R)* Length(R)
- ✓ 属性的特征值：指关系**R**中属性**A**不同的属性值个数，记为**Val(A)**
- ✓ 属性**A**的值域：记为**Dom(A)**
- ✓ 属性**A**的最大值和最小值：记为**Max(A)**和**Min(A)**

存取优化的理论基础

■关系运算的特征参数

假设：R、S为关系。

✓ 选择运算 ($S=\sigma_F(R)$)

选择度：

满足选择谓词F的元组与R元组总数之比，记为ρ

基数： Card（S）=ρ* Card（R）

关系的宽度：

Length（S）= Length（R）

➔ Length（S.A）= Length（R.A）

存取优化的理论基础

- ✓ 选择运算(中间关系大小)

选择度ρ的具体计算方法:

①等值比较

$S=\sigma_{A=X}(R)$，其中A是R的属性，X是常数。

则$\rho=1/Val(R, A)$

②非等值比较

$S=\sigma_{A>X}(R)$时： $\rho = (Max(A)-X)/(Max(A)-Min(A))$

$S=\sigma_{A<X}(R)$时： $\rho = (X-Min(A))/(Max(A)-Min(A))$

③不等比较

$S=\sigma_{A\neq X}(R)$时: $\rho= (Val(R, A)-1)/ Val(R, A)$

④多属性选择条件

$S=\sigma_{Ci\ AND\ Cj}(R)$时: $\rho= \rho_i*\rho_j$

$S=\sigma_{Ci\ OR\ Cj}(R)$时: $\rho =1-(1-\rho_i)(1-\rho_j)=(\rho_i+\rho_j -\rho_i *\rho_j)$

存取优化的理论基础

✓选择运算 ($S= \sigma_F (R)$)

不同值的个数：

a.设属性B不属于选择谓词F，其值均匀分布。

$$Val(S, B)=\begin{cases} Card(S) & 当Card(S)<Val(R, B)/2时 \\ (Card(S)+Val(R, B))/3 & 当Val(R, B)/2\leq Card(S)<2Val(R, B)时 \\ Val(R, B) & 当Card(S)\geq 2Val(R, B)时 \end{cases}$$

b.设属性B属于选择谓词F

若**B=X**（值），则：**Val（S，B）=1**

若**B**与选择谓词**F**相关且为关键字，

则：**Val（S，B）=ρ* Val（R，B）**

存取优化的理论基础

✓选择运算 ($S= \sigma_F (R)$)

✓举例： 设**Card(R)=100，Val(R,B)=70**

令ρ=0.1

则： Card(S)=ρ* Card(R) =0.1*100=10

∵Card(S)=10， Val(R,B)/2=35

∴Card(S)< Val(R,B)/2， Val(S,B)=Card(S)=10

令ρ=0.5

则： Card(S)=ρ* Card(R) =0.5*100=50

∵Card(S)=50 , Val(R,B)/2=70/2=35

∴Val(R,B)/2<Card(S)<2* Val(R,B),

Val(S,B)=(Card(S)+ Val(R,B))/3=40

存取优化的理论基础

- **投影运算 ($S=\pi_A(R)$)**
  - ✓ **基数：**

①如果投影涉及单个属性A：Card（S）= Val（R，A）

②如果A中包含关键字：Card（S）= Card（R）

③投影涉及多个属性，且有 $\prod_{A_i \in AttrList} Val(R, A_i) < Card(R)$

则 $Card(S) = \prod_{A_i \in AttrList} Val(R, A_i)$

  - ✓ **关系的宽度：**

Length（S）=∑Length（A_i）($A_i \in A$)

Size（S）= Card（S）* Length（S）

Size（S）< Size（R）

  - ✓ **不同值的个数：**

Val(S,A)= Val(R,A)

机械工业出版社
CHINA MACHINE PRESS

存取优化的理论基础

- 并、交与差(如果结果关系执行消除重复操作，则都只能计算出结果关系基数的上限和下限，因此通常采用平均值作为基数的估计值。)

  ✓ 基数：

  ①并运算

  如果不消除重复，则结果关系基数等于两个关系基数之和：

  Card(T)= Card(R)+ Card(S)

  如果消除重复，则结果关系基数最大可大至两个关系基数之和，最小可小至两个关系基数的大者，因此有：

  Max{Card(R), Card(S)}<Card(T)<Card(R)+ Card(S)

  在估计中使用中间值作为结果关系基数:

  (Max{Card(R), Card(S)}+Card(R)+ Card(S))/2

机械工业出版社
CHINA MACHINE PRESS

存取优化的理论基础

- 并、交与差
  - ✓ 基数：

②交运算

交运算的结果关系最小可以是空，最大可以等于两个关系的较小者，因此按取区间中间值的方法估计结果关系的基数为较小关系基数的一半：

Card(T)= Min{Card(R), Card(S)}/2

③差运算

对于两个关系的差运算**R-S**，其结果关系基数的区间为：

Card(R)-Card(S)<Card(T)<Card(R)

其中**Card(R)-Card(S)**在**S**包含**R**中所有元组时**Card(R)-Card(S)=0**。

结果关系基数：**Card(T)=(2Card(R)-Card(S))/2**

存取优化的理论基础

关系代数举例

- 笛卡尔连接

R

A	B	C
a1	b1	c1
a1	b2	c2
a2	b2	c1

S

A	B	C
a1	b2	c2
a1	b3	c2
a2	b2	c1

R×S

A	B	C	A	B	C
a1	b1	c1	a1	b2	c2
a1	b1	c1	a1	b3	c2
a1	b1	c1	a2	b2	c1
a1	b2	c2	a1	b2	c2
a1	b2	c2	a1	b3	c2
a1	b2	c2	a2	b2	c1
a2	b2	c1	a1	b2	c2
a2	b2	c1	a1	b3	c2
a2	b2	c1	a2	b2	c1

存取优化的理论基础

- 笛卡尔积
  - ✓基数:
    Card(T)= Card(R)* Card(S)
  - ✓元组的长度:
    Length(T)=Length(R)+ Length(S)
  - ✓属性不同值的数量:
    笛卡尔积结果关系中属性不同值的数量等于其原关系中 对应属性的不同取值的数量:
    Val(T, A)= Val(R, A) 或者
    Val(T, A)= Val(S, A)

存取优化的理论基础

关系代数举例

- 等值连接和自然连接

R

A	B	C
a1	b1	5
a1	b2	6
a2	b3	8
a2	b4	12

S

B	E
b1	3
b2	7
b3	10
b3	2
b5	2

$R\infty_{R.B=S.B}S$

A	R.B	C	S.B	E
a1	b1	5	b1	3
a1	b2	6	b2	7
a2	b3	8	b3	10
a2	b3	8	b3	2

$R\infty S$

A	B	C	E
a1	b1	5	3
a1	b2	6	7
a2	b3	8	10
a2	b3	8	2

存取优化的理论基础

■连接运算 S=R∞T，（R.a=T.b）

- ✓ 基数：存在Card（S）≤ Card（R）× Card（T）

  具体分下面几种情况：

- ✓ 基本计算形式（ρ为连接选择度）

$$\rho(R,S)=\frac{card(R\infty S)}{card(R)\bullet card(S)}$$

Card（S）=ρ* Card（R）* Card（T）

●若T以b为关键字，R中a为外来关键字

Card（S）= Card（R）

- ✓ 关系的宽度（a泛指连接属性）：

Length（S）= Length（R）+ Length（T）

Length（S.a）= Length（R.a）

通常：Card(R∞T)=Card(R)*Card(T)/max(Val(R,a),Val(T,a))

当Val(R,a)≤Val(T,a)，R中每个元组有1/Val(T,a)机会与T中元组连接

机械工业出版社
CHINA MACHINE PRESS

存取优化的理论基础

■连接运算 S=R∞T，（R.a=T.b）

✓不同值的个数：

● 设a为连接属性

Val(S,a)≤Min(Val(R,a), Val(T,a))

● 若c不为连接属性

Val（S，c）≤Val（R，c）(c为R的属性)

Val（S，c）≤Val（T，c）(c为T的属性)

存取优化的理论基础

- **半连接运算 S=R∝T，（R.a=T.b）**

  半连接操作是全连接操作的一种缩减。是一种导出操作，且具有不对称性。如：半连接操作（R∝T）是R与T自然连接后在R上的投影，

  描述为：$R \propto T = \pi_{Attr（R）}(R \infty T)$

  存在：Card（R∝T）≤Card（R） R∝T≠ T∝R

  - ✓ 基数： Card（S）=ρ* Card（R）ρ为半连接选择度
  - ✓ 关系的宽度：Length（S）= Length（R）
  - ✓ 不同值的个数：

    a.设a为连接属性

    Val（S，a）=ρ* Val（R，a）

    b.若c不为连接属性

    Val（S，c）≤Val（R，c）

半连接优化方法

- 对连接操作的优化有两种趋势：
- 一种为采用半连接技术，减少连接操作的操作数，以降低传输费用；
- 另一种为采用全连接技术，主要考虑局部代价。
- 一个系统需根据其目标综合确定其优化算法。

半连接优化方法

□半连接的作用

假设有雇员关系EMP和部门关系DEPT，

EMP：Card（EMP）=10000；DEPT：Card（DEPT）=100

其中，关系EMP保存在场地S1，关系DEPT保存在场地S2，如下图所示。

现有查询要求：在场地S2上查询部门名称和部门经理姓名，选择最优的执行策略，这里只考虑数据传输代价。查询的SQL语句如下：

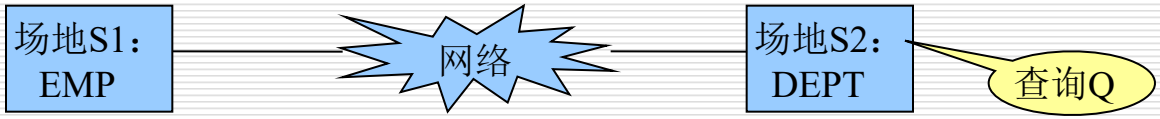
```
SELECT DNAME, ENAME
FROM DEPT, EMP
WHERE DEPT.MgrNO =EMP.ENO
```

关系代数表达式为：

$Q=\pi_{DNAME,\ ENAME}(DEPT\infty EMP)$

属性	ENO	ENAME	……
长度(B)	4	35	

属性	DNO	DNAME	MgrNo	…
长度(B)	4	35	4	



半连接优化方法

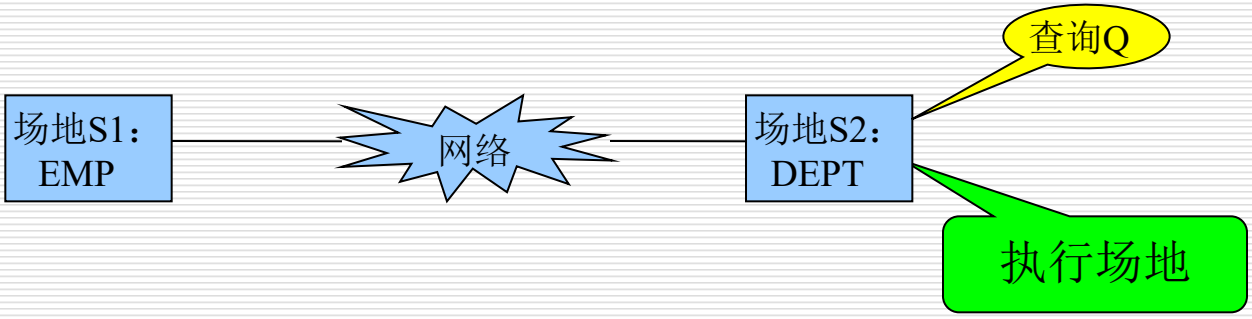
半连接的作用

下面通过三种查询策略分析其代价评估（**COST**）

策略1：执行场地设在S2

需将EMP的Eno和Ename属性传送到S2场地

COST = (Length(Eno)+Length(Ename))* Card(EMP)
= 39B*10000=39KB



半连接优化方法

半连接的作用

策略2：执行场地设在**S1**

需将**DEPT**的**Dname**和**Mgno**属性传送到**S1**场地，操作后，再将结果传回场地**S2**。设**R**为结果。

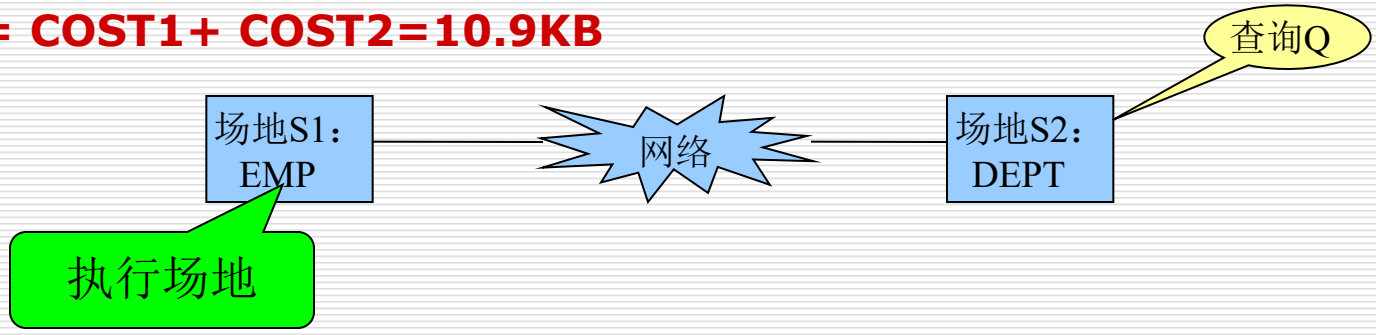
COST1=(Length(Dname)+Length(Mgno))*Card(DEPT)

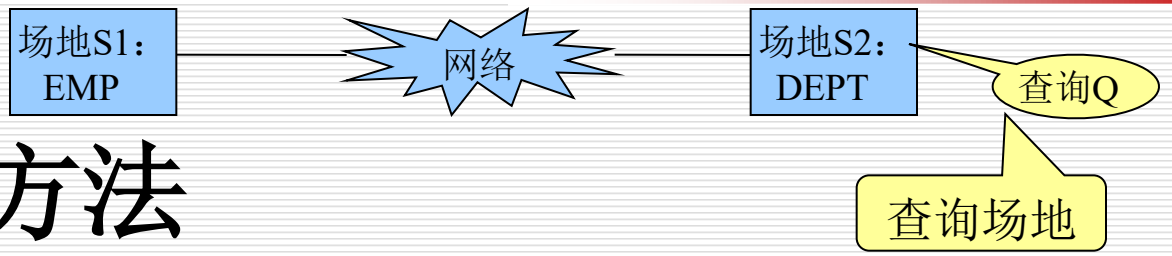
=39B*100=3.9KB

COST2=(Length(Dname)+Length(Ename))* Card(R)

=70B*100=7KB

COST= COST1+ COST2=10.9KB





半连接优化方法

□ 半连接的作用

策略3：采用半连接

①将DEPT的Mgno属性传送到S1场地，

即将D1=π_{Mgno}（DEPT）传送到S1场地。

COST1=Length(Mgno)*Card(DEPT)=4*100=0.4KB

②在场地S1。完成EMP与D1的连接，即实现E1=EMP∞D1，

则：Card（E1）=100。

将E1的属性Eno和Ename传到S2，

即将E2=$\pi_{Eno,\ Ename}$（E1）传到S2。

COST2=(Length(Eno)+Length(Ename))* Card(E1)

=39*100=3.9KB

③在场地S2上计算： R=DEPT∞ E2。

COST= COST1+ COST2=0.4+3.9=4.3 KB。

策略3相当于：EMP ∞ DEPT=（EMP ∝ DEPT）∞DEPT。

①②实现 ③实现

半连接优化方法

半连接优化算法

输入信息：位于不同场地上的两个关系R和S

输出信息：实现R∞S（R.A=S.B）

算法：（设S的尺寸小于R）

（1）在S所在场地上计算$S'=\pi_B(S)$;

（2）传送S′到R场地

（3）在R场地上计算R′=R∞S′=R∝S

（4）将R′传到S所在场地

（5）在S所在场地上计算

R′∞S =(R∝S)∞S= R∞S

半连接优化方法

□ 传输代价的比较

假设：

关系R和S分别在不同的场地上，C_0为启动代价，C_1为单位传输代价。

- 设：在S所在的场地上执行，则传输关系R实现R∞S的代价

$$C=C_0+C_1*(Length(R)*\ Card(R)\ )$$
$$=\ C_0+C_1*Size(R)$$

半连接优化方法

- 传输代价的比较

  采用半连接的传输代价（需传输S′和R′）

$C_{\propto} = C_{S'} + C_{R'}$

$= C_0 + C_1 * (Length(S') * Card(S')) +$

$C_0 + C_1 * (Length(R') * Card(R'))$

$= 2\ C_0 + C_1 * (Length(S') * Card(S') + Length(R') * Card(R'))$

$= 2\ C_0 + C_1 * (Size(S') + Size(R'))$

分析：如果有：$C_{\infty} \geq C_{\propto}$

则：$C_0 + C_1 * Size(R) \geq 2\ C_0 + C_1 * (Size(S') + Size(R'))$

$C_0 / C_1 + Size(S') + Size(R') \leq Size(R)$

枚举法优化技术

- 查询操作的代价评估：综合考虑局部代价和传输代价。
  - 若侧重传输代价，局部代价可以忽略不计时，采用半连接技术较好；
  - 侧重局部代价时，采用直接连接比采用半连接优越。
- 枚举法是基于直接连接的实现方法。

枚举法优化技术

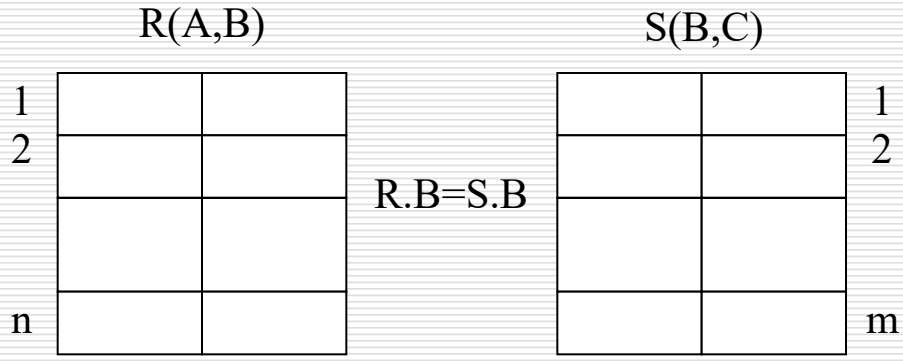
- 常见的直接连接算法主要有：
  - 嵌套循环连接算法(nest-loop)
  - 归并排序连接算法（merge-scan）
  - 哈希连接算法（Hash）
  - 基于索引的连接算法
- 对每种直接连接算法进行代价估计：考虑执行连接操作所需的磁盘I/O代价。
- 对磁盘I/O代价的估计主要依赖于数据库的特征参数。

枚举法优化技术

嵌套循环连接算法

- 是一种最简单的连接算法，原理是对连接操作的两个关系中的一个仅读取其元组一次，而对另一个关系中的元组将多次读取。
- 特点是可以用于任何大小的关系间的连接操作，不必受连接操作所分配的内存空间大小的限制。

假设有关系**R(A,B)**和**S(B,C)**，**Card(R)=n**，**Card(S)=m**，现在要执行两个关系在属性**B**上的连接操作.



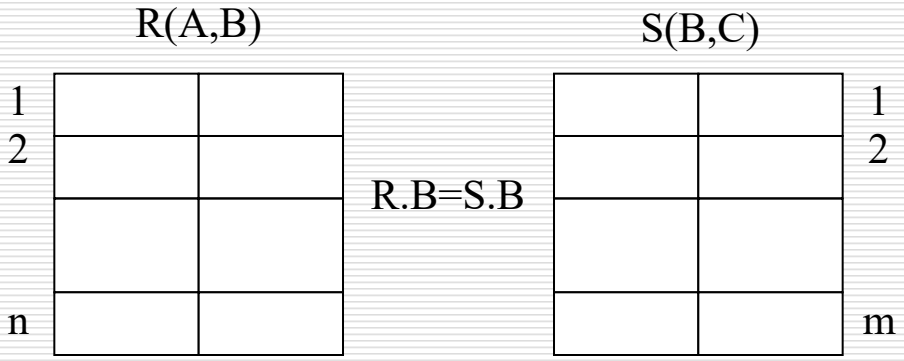
枚举法优化技术

- 基于元组的嵌套循环连接
  - 循环以关系中的元组为单位进行操作，具体的执行算法如下：

```
Result=∅ /*初始化结果集合*/
For each tuple s in S
  For each tuple r in R
    If r.B=s.B Then /*元组r和元组s满足连接条件*/
      Join r and s as tuple t;
      Output t into Result; /*输出连接结果元组*/
    End If
  End For
End For
Return Result
```

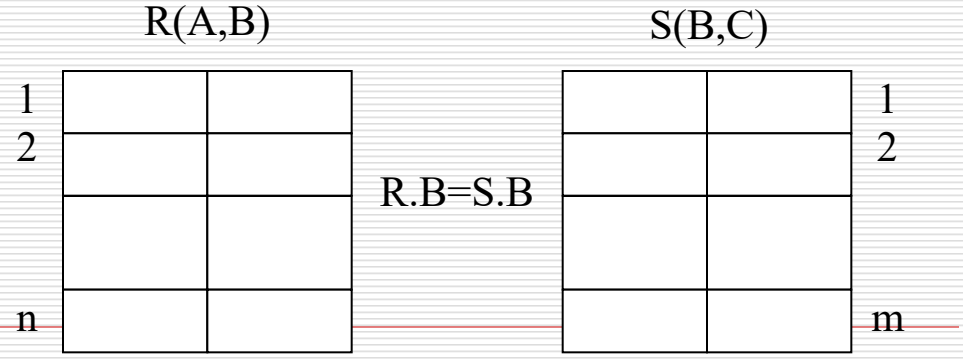
外关系S

内关系R



枚举法优化技术

- 基于元组的嵌套循环连接
  - 在执行嵌套循环连接时，仅对外关系进行1次读取操作，而对内关系则需要进行多次读取。
  - 如果不进行优化的话，执行代价很大，以磁盘IO代价计算最多可能多达Card(S)*Card(R) =mn次。
  - 通常对这种算法进行修改，一种方法是通过尽可能多地使用内存以减少磁盘IO次数；另一种方法是使用连接属性上的索引，以减小参与连接元组的数量。



枚举法优化技术

- 基于块的嵌套循环连接
  - 通过尽可能多的使用内存，减少读取元组所需的I/O次数。
  - 对连接操作的两个关系的访问均按块（也称为页面）进行，同时使用尽可能多的内存来存储嵌套循环中外关系的块。

假设：为连接操作分配的内存缓冲区大小为M个块，有Block(R)≥Block(S)≥M，

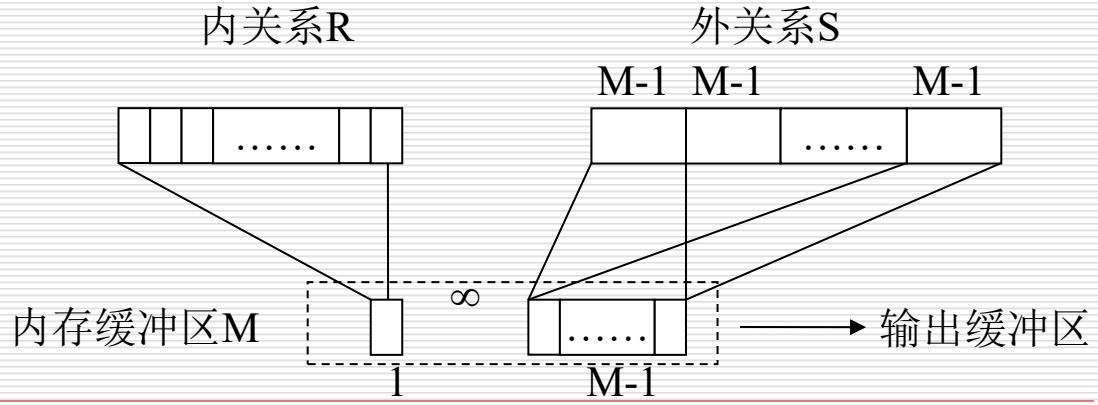
  - 实现过程：首先选取较小的关系作为外关系，本例即关系S。将1到M-1块分配给关系S,第M块分配给关系R。
  - 将外关系S按照M-1个块的大小分为多个子表，并重复地将这些子表读取到内存缓冲区中，用于重复地依次读取关系R的每一个块。
  - 对于内存缓冲区中元组的连接操作，先在M-1个块的外关系S元组的连接属性上构建查找结构，再从内关系R在内存中的块中取元组，通过查找结构与S中的元组连接。

枚举法优化技术

基于块的嵌套循环连接

算法：

```
Result=∅ /*初始化结果集合*/
Buffer=M /*内存缓冲区*/
For each M-1 in Block(S) /*每次从外关系S中读取M-1个块到内存缓冲区*/
  Read M-1 of Block(S) into Buffer;
  For each block in Block(R) /*每次从内关系R中读取1个块到内存缓冲区*/
    Join M-1 of Block(S) and 1 of Block(R) in Buffer;/*在内存中对元组执行连接*/
    Output t into Result;
  End For
End For
Return Result
```



枚举法优化技术

基于块的嵌套循环连接方法的代价估计

假设R和S占用的块分别为Block(R)和Block(S)，M为内存缓冲区大小（块数）。一次I/O可读取一个数据块。则共需读取关系S的块数为Block(S)，共需读取R的块数为Block(R)* Block(S)/(M-1)。

$$总I/O次数 = Block(S) + \frac{Block(S)}{M-1} * Block(R)$$

说明：

如果连接关系Block(R)*Block(S)的值远远大于内存缓冲区大小M时，认为连接的代价近似等于Block(R)*Block(S)。

这种算法能够适用于任意大小的关系之间的连接执行，因此基于块的嵌套循环连接算法依然广泛应用于现有的数据库系统中。

枚举法优化技术

- 嵌套循环连接方法举例
  - 假设连接的两个关系R和S，Block(R)=2000，Block(S)=500，内存缓冲区M=51。则：
    外关系为S
    内存分配：M-1=50分配给S，再循环读取关系R的2000个块。
    根据公式：

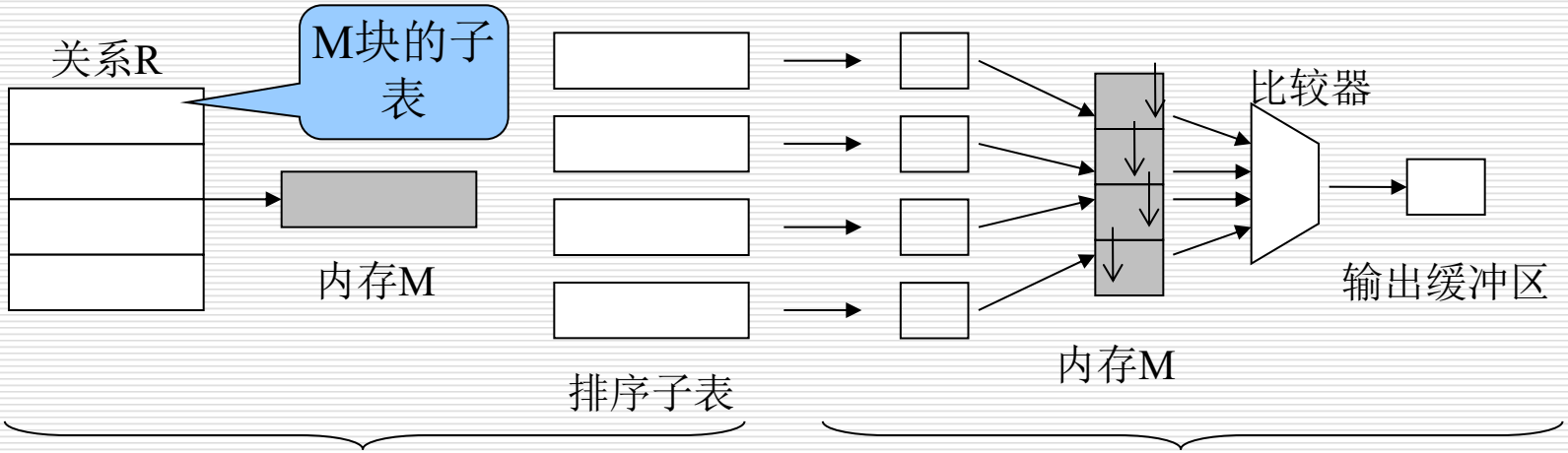
$$Block(S)+\frac{Block(S)}{M-1}*Block(R)$$

    连接的执行代价为：
    500/(51-1)*2000+500=20500次磁盘IO。

机械工业出版社
CHINA MACHINE PRESS

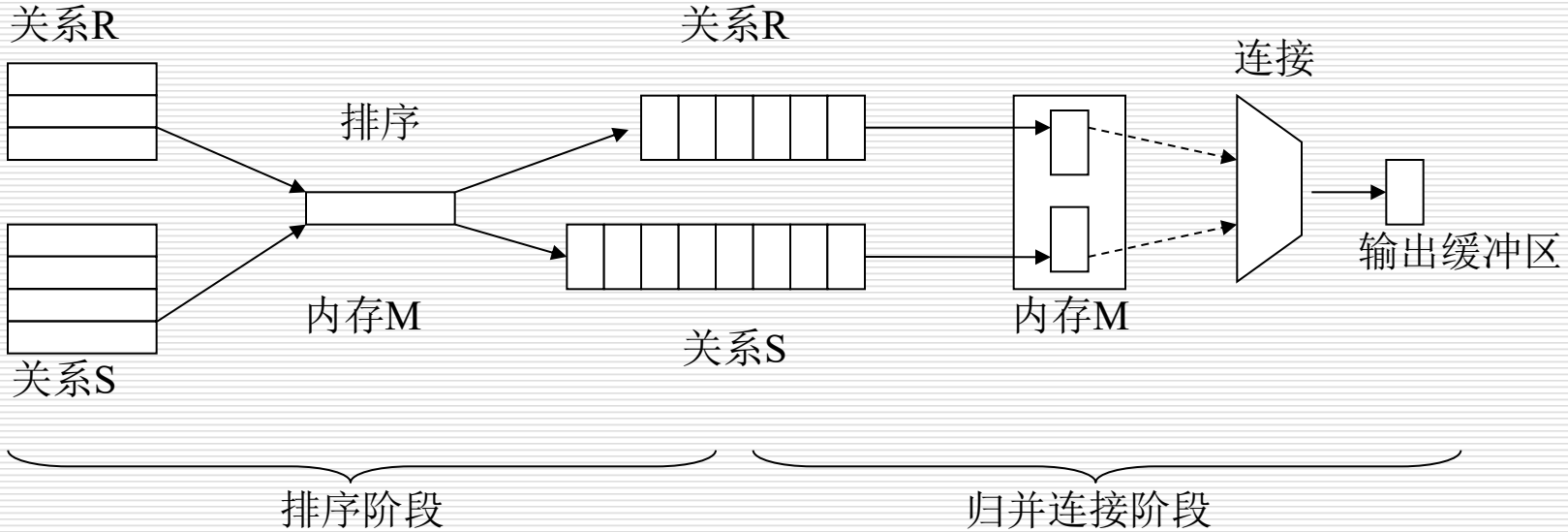
枚举法优化技术

- 基于排序的连接算法
  - 首先将两个关系按照连接属性进行排序(分段排序+归并)
  - 然后按照连接属性的顺序扫描两个关系，同时对两个关系中的元组执行连接操作。
- 简单的归并排序算法的执行过程



枚举法优化技术

- 简单基于排序的连接算法
  - 对已经按照连接属性排序的两个关系，按照顺序读取关系中的块到内存中执行连接操作。



- 执行代价：$C_{join} = C_{排序I/O}+C_{连接IO} = C_{分段排序I/O}+C_{归并I/O}+C_{连接I/O}$

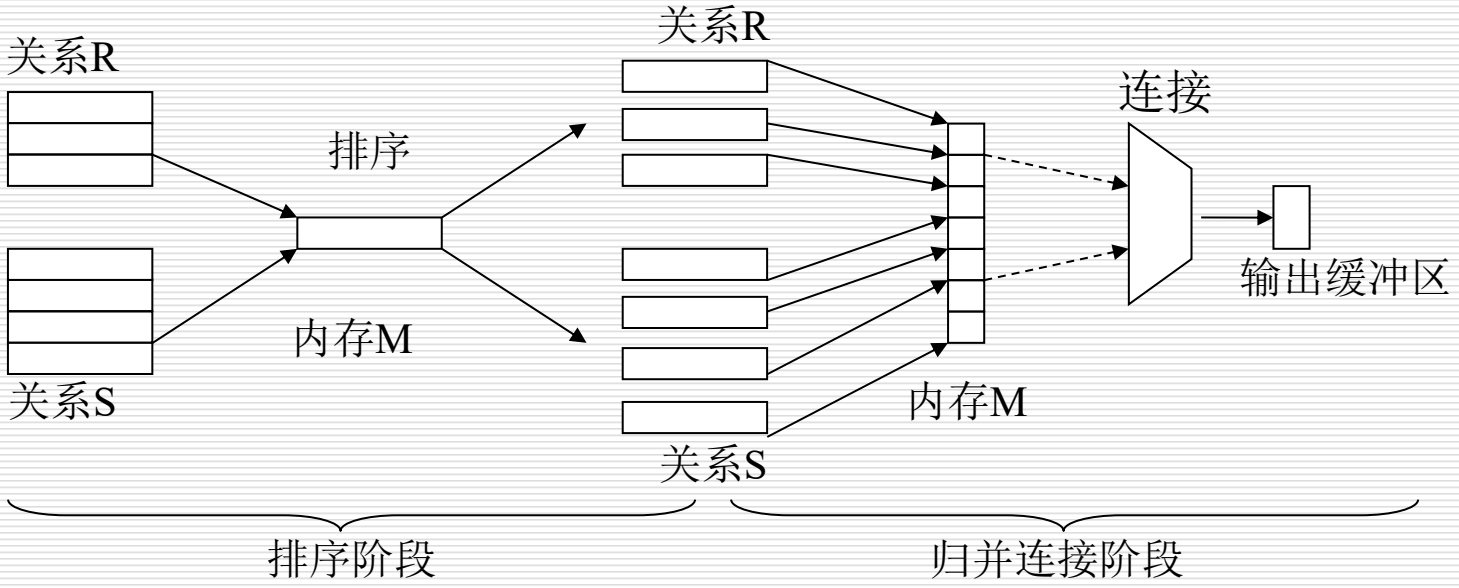
$= 2(Block(R)+ Block(S)) +2(Block(R)+ Block(S)) +(Block(R)+ Block(S))$

$=5(Block(R)+ Block(S))$

枚举法优化技术

归并排序连接算法

- 其思想是将排序的第二阶段与归并连接阶段合并，这样可以节省一次对关系的读写操作。



- 执行代价为：$C_{join} = 3(Block(R) + Block(S))$

枚举法优化技术

- 哈希连接算法
  - 使用同一个哈希函数，对进行连接的两个关系**R**和**S**中的元组的连接属性值进行散列，具有相同属性值的元组会出现在相同**Hash**数值的桶中，然后对对应的桶中的元组执行连接
  - 执行代价为：C_{join}=3(Block(R)+ Block(S))
    - ✓哈希代价：一次读写2(Block(R)+ Block(S))
    - ✓桶连接：若一个桶能全部装入内存缓冲区块中，需要一次读取操作(Block(R)+ Block(S)

枚举法优化技术

- 连接关系的传输方法

  当参与连接的两个关系在不同场地时，需要将它们传输到同一场地执行连接操作。

  - 全体传输

    $C_{tran} = [(Card(R)*Length(R))/m]*C_{mes}$

    其中m为报文容量，C_{mes}为报文的单位传输代价

  - 按需传输

    $C_{tran} = l* C_{mes} +[(Card(R')*Length(R'))/m]*C_{mes}$

    其中l为请求报文，R'为需要传输的关系